

几何

二维几何

```
1  const double eps = 1e-8;
2  //const int eps = 0;
3  using T = long double;
4  constexpr T PI = 3.14159265358979323841;
5
6  struct Point
7  {
8      T x = 0, y = 0;
9      int id = 0;
10     Point(T _x = 0, T _y = 0) : x(_x), y(_y) {}
11
12     //向量代数运算
13     friend bool operator == (const Point & x, const Point & y) { return (abs(x.x -
14     y.x) <= eps && abs(x.y - y.y) <= eps); }
15     bool operator < (const Point &a) const { if (abs(x - a.x) <= eps) return y < a.y
16     - eps; return x < a.x - eps; }
17     bool operator > (const Point &a) const { return !(*this < a || *this == a); }
18     friend Point operator + (const Point & x, const Point & y) { return {x.x + y.x,
19     x.y + y.y}; }
20     friend Point operator - (const Point & x, const Point & y) { return {x.x - y.x,
21     x.y - y.y}; }
22     friend Point operator * (const Point & x, const T k) { return {k * x.x, k * x.y};
23     }
24     friend Point operator * (const T k, const Point & x) { return {k * x.x, k * x.y};
25     }
26     friend Point operator / (const Point & x, const T k) { return {x.x / k, x.y / k};
27     }
28     friend T operator * (const Point & x, const Point & y) { return x.x * y.x + x.y *
29     y.y; }
30     friend T operator ^ (const Point & x, const Point & y) { return x.x * y.y - x.y *
31     y.x; }
32     Point operator - () const { return {-x, -y}; }
33     Point& operator += (const Point & x) { return *this = *this + x; }
34     Point& operator -= (const Point & x) { return *this = *this - x; }
35     Point& operator *= (T k) { return *this = *this * k; }
36
37     //长度和旋转
38     T len2() const { return (*this) * (*this); } //模长平方
39     T len2(Point a) const { return ((*this) - a).len2(); } //两点距离平方
40     T len() const { return (T)sqrt(len2()); } //模长
41     T len(Point a) const { return ((*this) - a).len(); } //两点距离
42     T ang(const Point &a) const { return acos1(max(-1.01, min(1.01, ((*this) * a) /
43     (len() * a.len())))); } // 向量夹角
44     Point rot90() const { return {-y, x}; }
45     Point rot(const T rad) const { return {x * cos(rad) - y * sin(rad), x * sin(rad)
46     + y * cos(rad)}; } // 逆时针旋转（给定角度）
47     Point rot(const T cosr, const T sinr) const { return {x * cosr - y * sinr, x *
48     sinr + y * cosr}; } // 逆时针旋转（给定角度的正弦与余弦）
```

```

36 //to_left测试(向量传入向量): 1表示在左侧, 0表示共线, -1表示右侧
37 int toleft(Point p) const
38 {
39     T ans = (*this) ^ p;
40     return (ans > eps) - (ans < -eps);
41 }
42 // 象限判断 0:原点 1:x轴正 2:第一象限 3:y轴正 4:第二象限 5:x轴负 6:第三象限 7:y轴负 8:第四
象限
43 int quad() const
44 {
45     if (abs(x) <= eps && abs(y) <= eps) return 0;
46     if (abs(y) <= eps) return x > eps ? 1 : 5;
47     if (abs(x) <= eps) return y > eps ? 3 : 7;
48     return y > eps ? (x > eps ? 2 : 4) : (x > eps ? 8 : 6);
49 }
50 };
51 // bool argcmp(const Point &a,const Point &b) { return (a^b)>eps; } 半平面极角排序
52
53 struct Argcmp
54 {
55     Point o;
56     Argcmp(Point p = {0, 0}) : o(p) {}
57     bool operator()(const Point &a,const Point &b) const
58     {
59         const int qa = (a - o).quad(), qb = (b - o).quad();
60         if (qa != qb)
61             return qa < qb;
62         const auto t = (a - o) ^ (b - o);
63         // if (abs(t) <= eps) return (a - o) * (a - o) < (b - o) * (b - o) - eps;//
不同长度的向量需要分开
64         return t > eps;
65     }
66 };
67 // 全平面极角排序
68
69 struct Line
70 {
71     Point p, v;
72     Line() {}
73     Line(Point a, Point b) { p = a, v = b; }
74
75     bool operator<(const Line &a) const // 半平面交算法定义的排序
76     {
77         if (abs(v ^ a.v) <= eps && v * a.v >= -eps)
78             return toleft(a.p) == -1;
79         return Argcmp()(v,a.v);
80     }
81
82     bool isConnect(const Line &a) const { return abs(v ^ a.v) > eps; }
83     int toleft(const Point &a) const {return v.toleft(a - p);} // to_left测试(线段传入
点)
84 //直线相关计算
85 T dis(const Point &a) const { return abs(v ^ (a - p)) / v.len(); } //点到直线距离

```

```

86     Point proj(const Point & a) const {return p + v * ((v * (a - p)) / (v * v)); }
    //点在直线上的投影
87     Point inter(const Line & a) const //直线交点
88     {
89         assert(isConnect(a));
90         return p + v * ((a.v ^ (p - a.p)) / (v ^ a.v));
91     }
92 };
93 struct Segment
94 {
95     Point a, b;
96     Segment(){}
97     Segment(Point _a, Point _b) : a(_a), b(_b) {}
98
99     bool operator < (const Segment & s) const { return make_pair(a, b) <
make_pair(s.a, s.b); }
100
101     //判定相关
102     // 判断点是否在线段上
103     // -1 点在线段端点 | 0 点不在线段上 | 1 点严格在线段上
104     int is_on(Point p) const
105     {
106         if(p == a || p == b)
107             return -1;
108         return (b - a).toleft(p - a) == 0 && (p - b) * (p - a) < -eps;
109     }
110     // 判断线段直线是否相交
111     // -1 直线经过线段端点 | 0 线段和直线不相交 | 1 线段和直线严格相交
112     int is_inter(const Line &l) const
113     {
114         if (l.toleft(a) == 0 || l.toleft(b) == 0)
115             return -1;
116         return l.toleft(a) != l.toleft(b);
117     }
118     // 判断两线段是否相交
119     // -1 在某一线段端点处相交 | 0 两线段不相交 | 1 两线段严格相交
120     int is_inter(const Segment &s) const
121     {
122         if (is_on(s.a) || is_on(s.b) || s.is_on(a) || s.is_on(b)) return -1;
123         const Line l{a,b-a},ls{s.a,s.b-s.a};
124         return l.toleft(s.a)*l.toleft(s.b)==-1 && ls.toleft(a)*ls.toleft(b)==-1;
125     }
126
127     //距离相关
128     // 点到线段距离（必须用浮点数）
129     T dis(const Point &p) const
130     {
131         if ((p - a) * (b - a) < -eps || (p - b) * (a - b) < -eps)
132             return min(a.len(p), b.len(p));
133         const Line l(a, b - a);
134         return l.dis(p);
135     }
136     // 两线段间距离（必须用浮点数）

```

```

137     T dis(const Segment &s) const
138     {
139         if (is_inter(s)) return 0;
140         return min({dis(s.a),dis(s.b),s.dis(a),s.dis(b)});
141     }
142 };
143 struct Polygon
144 {
145     vector<Point> point;
146     Polygon() {}
147     Polygon(vector<Point> p) : point(p) {}
148
149     //获取下一个和上一个下标
150     int nxt(const int i) const { return i == point.size() - 1 ? 0 : i + 1; }
151     int pre(const int i) const { return i == 0 ? point.size() - 1 : i - 1; }
152     //求转数: 判断点是否在一般多边形上或内外部: bool范围是否在边上, int计数回转数
153     //https://ac.nowcoder.com/acm/contest/27249/B
154     pair<bool,int> winding(const Point &a) const
155     {
156         int cnt = 0;
157         for (int i = 0; i < point.size(); ++i)
158         {
159             const Point u = point[i], v = point[nxt(i)];
160             if (Segment(u, v).is_on(a))
161                 return {true, 0};
162             if (abs(u.y - v.y) <= eps)
163                 continue;
164             const Line uv = {u, v - u};
165             if (u.y < v.y - eps && uv.toleft(a) <= 0)
166                 continue;
167             if (u.y > v.y + eps && uv.toleft(a) >= 0)
168                 continue;
169             if (u.y < a.y - eps && v.y >= a.y - eps)
170                 cnt++;
171             if (u.y >= a.y - eps && v.y < a.y - eps)
172                 cnt--;
173         }
174         return {false,cnt};
175     }
176     // 多边形面积的两倍: 可用于判断点的存储顺序是顺时针或逆时针
177     T area() const
178     {
179         T sum = 0;
180         for(int i = 0; i < point.size(); ++i)
181             sum += point[i] ^ point[nxt(i)];
182         return sum;
183     }
184     // 多边形的周长
185     T circ() const
186     {
187         T sum = 0;
188         for (int i = 0; i < point.size(); ++i)
189             sum += point[i].len(point[nxt(i)]);

```

```

190         return sum;
191     }
192 };
193 struct Convex : Polygon
194 {
195     Convex(){}
196     Convex(vector<Point> p) : Polygon(p) {}
197
198     // 闵可夫斯基和
199     Convex operator + (const Convex & c) const
200     {
201         vector<Segment> e1(point.size()), e2(c.point.size()), edge(point.size() +
198 c.point.size());
202         vector<Point> res;
203         res.reserve(point.size() + c.point.size());
204         const auto cmp = [](const Segment & u, const Segment & v)
205         { return Argcmp()(u.b - u.a, v.b - v.a); };
206         for(int i = 0; i < point.size(); ++i)
207             e1[i] = Segment(point[i], point[nxt(i)]);
208         for(int i = 0; i < c.point.size(); ++i)
209             e2[i] = Segment(c.point[i], c.point[c.nxt(i)]);
210         rotate(e1.begin(), min_element(e1.begin(), e1.end(), cmp), e1.end());
211         rotate(e2.begin(), min_element(e2.begin(), e2.end(), cmp), e2.end());
212         merge(e1.begin(), e1.end(), e2.begin(), e2.end(), edge.begin(), cmp);
213         const auto check = [](const vector<Point> & res, const Point & u)
214         {
215             const auto back1 = res.back(), back2 = *prev(res.end(), 2);
216             return (back1 - back2).toleft(u - back1) == 0 && (back1 - back2) * (u -
217 back1) >= -eps;
218         };
219         auto u = e1[0].a + e2[0].a;
220         for(const auto & v : edge)
221         {
222             while(res.size() > 1 && check(res, u))
223                 res.pop_back();
224             res.push_back(u);
225             u = u + v.b - v.a;
226         }
227         if(res.size() > 1 && check(res, res[0]))
228             res.pop_back();
229         return res;
230     }
231     Convex operator - (const Convex & c) const
232     {
233         Convex nc = c;
234         for(auto & p : nc.point)
235             p = -p;
236         return (*this) + nc;
237     }
238
239     // 旋转卡壳: O(n)求凸包直径平方
240     // https://ac.nowcoder.com/acm/contest/28513/B
241     // https://www.luogu.com.cn/problem/A452

```

```

241     T rotcaliper() const
242     {
243         if(point.size() <= 1)
244             return 0;
245         if(point.size() == 2)
246             return point[0].len2(point[1]);
247         T ret = 0;
248         const auto area = [](const Point &u, const Point &v, const Point &w) { return
249 (w - u) ^ (w - v); };
250         int n = point.size(), r = 1;
251         for(int i = 0; i < n; ++i)
252         {
253             ret = max({ret, point[i].len2(point[r]), point[nxt(i)].len2(point[r])});
254             while(area(point[i], point[nxt(i)], point[r]) <= area(point[i],
255 point[nxt(i)], point[nxt(r)]))
256             {
257                 r = nxt(r);
258                 ret = max({ret, point[i].len2(point[r]),
259 point[nxt(i)].len2(point[r])});
260             }
261         }
262         return ret;
263     }
264
265     // 判断点是否在凸多边形内, 复杂度 O(logn)
266     // https://ac.nowcoder.com/acm/contest/28770/A
267     // -1 点在多边形边上 | 0 点在多边形外 | 1 点在多边形内
268     int is_in(const Point &p) const
269     {
270         int n = point.size();
271         if (n == 1)
272             return p == point[0] ? -1 : 0;
273         if (n == 2)
274             return Segment(point[0], point[1]).is_on(p) ? -1 : 0;
275         if (p == point[0])
276             return -1;
277         if ((point[1] - point[0]).toleft(p - point[0]) == -1 || (point.back() -
278 point[0]).toleft(p - point[0]) == 1)
279             return 0;
280         const auto cmp = [&](const Point &u, const Point &v)
281         { return (u - point[0]).toleft(v - point[0]) == 1; };
282         const int i = lower_bound(point.begin() + 1, point.end(), p, cmp) -
283 point.begin();
284         if(i == 1)
285             return Segment(point[0], point[i]).is_on(p) ? -1 : 0;
286         if (i == n - 1 && Segment(point[0], point[i]).is_on(p))
287             return -1;
288         if (Segment(point[i - 1], point[i]).is_on(p))
289             return -1;
290         return (point[i] - point[i - 1]).toleft(p - point[i - 1]) > 0;
291     }
292
293     // 凸多边形关于某一方向的极点

```

```

289 // 复杂度  $O(\log n)$ 
290 // 参考资料: https://codeforces.com/blog/entry/48868
291 template<typename F> int extreme(const F & dir) const
292 {
293     const auto check = [&](const int i)
294     { return dir(point[i]).toleft(point[this->nxt(i)] - point[i]) >= 0; };
295     const auto dir0 = dir(point[0]);
296     const auto check0 = check(0);
297     if (!check0 && check(point.size()-1))
298         return 0;
299     const auto cmp = [&](const Point & v)
300     {
301         const int vi = &v - point.data();
302         if(vi == 0)
303             return 1;
304         const auto checkv = check(vi);
305         const auto t = dir0.toleft(v - point[0]);
306         if (vi == 1 && checkv == check0 && t==0)
307             return 1;
308         return checkv ^ (checkv == check0 && t <= 0);
309     };
310     return partition_point(point.begin(), point.end(), cmp) - point.begin();
311 }
312
313 // 过凸多边形外一点求凸多边形的切线, 返回切点下标
314 // 复杂度  $O(\log n)$ 
315 // 必须保证点在多边形外
316 pair<int, int> tangent(const Point &a) const
317 {
318     if(point.size() == 1)
319         return {0, 0};
320     if(point.size() == 2)
321         return {0, 1};
322     const int u = extreme([&](const Point &u){ return u-a; });
323     const int v = extreme([&](const Point &u){ return a-u; });
324     return {u, v};
325 }
326
327 // 求平行于给定直线的凸多边形的切线, 返回切点下标
328 // 复杂度  $O(\log n)$ 
329 pair<int, int> tangent(const Line &a) const
330 {
331     const int u = extreme([&](const Point& ) { return a.v; });
332     const int v = extreme([&](const Point& ) { return -a.v; });
333     return {u, v};
334 }
335 };
336
337
338 // 点集的凸包: Andrew 算法, 复杂度  $O(n \log n)$ , 保证结果是逆时针
339 // https://ac.nowcoder.com/acm/contest/28513/A
340 // https://www.luogu.com.cn/problem/P2742
341 Convex convexhull(vector<Point> p)

```

```

342 {
343     vector<Point> st;
344     if (p.empty())
345         return Convex(st);
346     if (p.size() == 1)
347         return Convex(p);
348     sort(p.begin(), p.end());
349     const auto check = [](const vector<Point> & st, const Point & u)
350     {
351         const auto back1 = st.back(), back2 = *prev(st.end(), 2);
352         return (back1 - back2).toleft(u - back1) <= 0;
353     };
354     for(const Point & u : p)
355     {
356         while (st.size() > 1 && check(st, u))
357             st.pop_back();
358         st.push_back(u);
359     }
360     int k = st.size();
361     p.pop_back();
362     reverse(p.begin(), p.end());
363     for(const Point & u : p)
364     {
365         while(st.size() > k && check(st, u))
366             st.pop_back();
367         st.push_back(u);
368     }
369     st.pop_back();
370     return Convex(st);
371 }
372
373 // 点集中面积最大最小的三角形，注意返回的面积两倍
374 // 这里使用极角排序和扫描线， $O(n^2 \log n)$ 
375 // https://ac.nowcoder.com/acm/contest/28513/C
376 pair<T,T> mnmTriangle(const vector<Point> & ps)
377 {
378     if(ps.size() <= 2)
379         return {0, 0};
380     vector<pii> dir;
381     T mx = 0, mn = INF;
382     for(int i = 0; i < ps.size(); ++i)
383     {
384         for(int j = 0; j < ps.size(); ++j)
385         {
386             if(i == j)
387                 continue;
388             if(ps[i] == ps[j])
389                 mn = 0;
390             else if(i != j)
391                 dir.push_back({i, j});
392         }
393     }
394     sort(dir.begin(), dir.end(), [&](const pii & x, const pii & y)

```



```

395     {
396         Point u = ps[x.first] - ps[x.second];
397         Point v = ps[y.first] - ps[y.second];
398         return Argcmp()(u, v);
399     });
400     vector<int> vx(ps.size()), ivx(ps.size());
401     for(int i = 0; i < ps.size(); ++i)
402         vx[i] = i;
403     sort(vx.begin(), vx.end(), [&](const int & x, const int & y)
404     { return ps[x].y > ps[y].y; });
405     for(int i = 0; i < ps.size(); ++i)
406         ivx[vx[i]] = i;
407     for(const auto & [u, v] : dir)
408     {
409         int x = ivx[u], y = ivx[v];
410         int l = min(x, y), r = max(x, y);
411         if(l > 0)
412             mn = min(mn, abs((ps[vx[l] - 1]] - ps[u]) ^ (ps[vx[l] - 1]] - ps[v])));
413         else if(r < ps.size() - 1)
414             mn = min(mn, abs((ps[vx[r + 1]] - ps[u]) ^ (ps[vx[r + 1]] - ps[v])));
415         mx = max({mx, abs((ps[vx.front()] - ps[u]) ^ (ps[vx.front()] - ps[v])),
416             abs((ps[vx.back()] - ps[u]) ^ (ps[vx.back()] - ps[v]))});
417         swap(vx[x], vx[y]);
418         ivx[u] = y, ivx[v] = x;
419     }
420     return {mn, mx};
421 }
422 // 仅获取点集的面积最大三角形
423 // 这里使用凸包和旋转卡壳, O(n^2)
424 T mxTriangle(const vector<Point> & ps)
425 {
426     Convex plg = convexhull(ps);
427     vector<Point> nps = plg.point;
428     if(nps.size() <= 2)
429         return 0;
430     const auto area = [](const Point & u, const Point & v, const Point & w)
431     { return abs((w - u) ^ (w - v)); };
432     int n = nps.size();
433     T ret = 0;
434     for(int i = 0; i < n; ++i)
435     {
436         int r = (i + 1) % n;
437         for(int j = (i + 1) % n; j != i; j = (j + 1) % n)
438         {
439             ret = max(ret, area(nps[i], nps[j], nps[r]));
440             while(area(nps[i], nps[j], nps[r]) <= area(nps[i], nps[j], nps[(r + 1) %
441 n]))
442             {
443                 r = (r + 1) % n;
444                 ret = max(ret, area(nps[i], nps[j], nps[r]));
445             }
446         }
447     }
448 }

```

```

446     return ret;
447 }
448
449 // 半平面交 排序增量法, 复杂度  $O(n\log n)$ 
450 // 输入与返回值都是用直线表示的半平面集合
451 // https://www.luogu.com.cn/problem/A452
452 // https://ac.nowcoder.com/acm/contest/29764/A
453 vector<Line> halfinter(vector<Line> l, const T lim = 1e18)
454 {
455     const auto check = [](const Line & a, const Line & b, const Line & c)
456     { return a.toleft(b.inter(c)) < 0; };
457     /* 无精度误差的方法, 但注意取值范围会扩大到三次方
458     const auto check= [](const Line &a, const Line &b, const Line &c)
459     {
460         const Point p=a.v*(b.v^c.v), q=b.p*(b.v^c.v)+b.v*(c.v^(b.p-c.p))-a.p*
461         (b.v^c.v);
462         return p.toleft(q)<0;
463     };
464     */
465     l.push_back({{-lim, 0}, {0, -1}}), l.push_back({{0, -lim}, {1, 0}});
466     l.push_back({{lim, 0}, {0, 1}}), l.push_back({{0, lim}, {-1, 0}});
467     sort(l.begin(), l.end());
468     deque<Line> q;
469     for(int i = 0; i < l.size(); ++i)
470     {
471         if(i > 0 && l[i - 1].v.toleft(l[i].v)==0 && l[i - 1].v*l[i].v > eps)
472             continue;
473         while(q.size() > 1 && check(l[i], q.back(), q[q.size()-2]))
474             q.pop_back();
475         while(q.size() > 1 && check(l[i], q[0], q[1]))
476             q.pop_front();
477         if(!q.empty() && q.back().v.toleft(l[i].v) <= 0)
478             return vector<Line>();
479         q.push_back(l[i]);
480     }
481     while(q.size() > 1 && check(q[0], q.back(), q[q.size()-2]))
482         q.pop_back();
483     while(q.size() > 1 && check(q.back(), q[0], q[1]))
484         q.pop_front();
485     return vector<Line>(q.begin(), q.end());
486 }
487
488 // 平面最近点对 扫描线 复杂度  $O(n\log n)$ 
489 // https://ac.nowcoder.com/acm/contest/52826/C
490 // https://www.luogu.com.cn/problem/P7883
491 T closest_pair(vector<Point> points)
492 {
493     sort(points.begin(), points.end());
494     const auto cmpy = [](const Point &a, const Point &b)
495     {
496         if(abs(a.y - b.y) <= eps)
497             return a.x < b.x - eps;
498         return a.y < b.y - eps;
499     };

```

```

498     };
499     multiset<Point, decltype(cmpy)> s{cmpy};
500     T ans = INF;
501     for(int i = 0, l = 0; i < points.size(); ++i)
502     {
503         const T sqans = sqrtl(ans) + 1;
504         while(l < i && points[i].x - points[l].x >= sqans)
505             s.erase(s.find(points[l++]));
506         for (auto it = s.lower_bound(Point{-INF, points[i].y-sqans});
507             it != s.end() && it->y-points[i].y<=sqans; ++it)
508             ans = min(ans, points[i].len2(*it));
509         s.insert(points[i]);
510     }
511     return ans;
512 }
513
514 // 判断多条线段是否有交点
515 // 扫描线, 复杂度 O(nlogn)
516 bool segs_inter(const vector<Segment> &segs)
517 {
518     if(segs.empty())
519         return false;
520     using seq_t = tuple<T, int, Segment>; // x坐标 出入点 线段
521     const auto seqcmp = [](const seq_t & u, const seq_t & v)
522     {
523         const auto [u0, u1, u2] = u;
524         const auto [v0, v1, v2] = v;
525         if (abs(u0 - v0) <= eps)
526             return make_pair(u1, u2) < make_pair(v1, v2);
527         return u0 < v0 - eps;
528     };
529     vector<seq_t> seq;
530     for(auto seg : segs)
531     {
532         if (seg.a.x > seg.b.x + eps)
533             swap(seg.a, seg.b);
534         seq.push_back({seg.a.x, 0, seg});
535         seq.push_back({seg.b.x, 1, seg});
536     }
537     sort(seq.begin(), seq.end(), seqcmp);
538     T x_now;
539     const auto cal= [](const Segment &u, const T x)
540     {
541         if(abs(u.a.x - u.b.x) <= eps)
542             return u.a.y;
543         if(x < u.a.x - eps)
544             return u.a.y;
545         if(x > u.b.x + eps)
546             return u.b.y;
547         return ((x - u.a.x) * (u.b.y - u.a.y) + u.a.y * (u.b.x - u.a.x)) / (u.b.x -
u.a.x);
548     };
549     auto cmp=[&](const Segment & u, const Segment & v)

```

```

550     { return cal(u, x_now) < cal(v, x_now) - eps; };
551     multiset<Segment, decltype(cmp)> s{cmp};
552     for(const auto [x, o, seg] : seq)
553     {
554         x_now = x;
555         const auto it = s.lower_bound(seg);
556         if(o == 0)
557         {
558             if(it != s.end() && seg.is_inter(*it))
559                 return true;
560             if(it != s.begin() && seg.is_inter(*prev(it)))
561                 return true;
562             s.insert(seg);
563         }
564         else
565         {
566             if(next(it) != s.end() && it != s.begin() &&
150 (*prev(it)).is_inter(*next(it)))
567                 return true;
568             s.erase(it);
569         }
570     }
571     return false;
572 }
573
574 // 多边形面积并
575 // 轮廓积分, 复杂度  $O(n^2 \log n)$ ,  $n$  为边数
576 // 注意 0-base, 并保证多边形为逆时针顺序
577 // ans[i] 表示被至少覆盖了  $i + 1$  次的区域的面积
578 // https://ac.nowcoder.com/acm/contest/36088/H
579 // https://www.luogu.com.cn/problem/P13338
580 vector<T> area_union(const vector<Polygon> & polys)
581 {
582     const int n = polys.size();
583     vector<vector<pair<Point, Point>>> segs(n);
584     const auto check = [](const Point & p, const Segment & seg)
585     { return !((p < seg.a && p < seg.b) || (p > seg.a && p > seg.b));};
586
587     auto cut_edge = [&](const Segment & u, const int i)
588     {
589         const Line L1{u.a, u.b - u.a};
590         vector<pair<Point, int>> evt;
591         evt.push_back({u.a, 0}), evt.push_back({u.b, 0});
592         for (int j = 0; j < polys.size(); ++j)
593         {
594             if(i == j)
595                 continue;
596             const auto & pj = polys[j];
597             for (int k = 0; k < pj.point.size(); ++k)
598             {
599                 const Segment v = {pj.point[k], pj.point[pj.nxt(k)]};
600                 if(L1.toleft(v.a) == 0 && L1.toleft(v.b) == 0)
601                     {

```

```

602         evt.push_back({v.a, 0});
603         evt.push_back({v.b, 0});
604     }
605     else if(v.is_inter(L1))
606     {
607         const Line L2{v.a, v.b - v.a};
608         const Point c = L1.inter(L2);
609         if(L1.toleft(v.a) < 0 && L1.toleft(v.b) >= 0)
610             evt.push_back({c, -1});
611         else if(L1.toleft(v.a) >= 0 && L1.toleft(v.b) < 0)
612             evt.push_back({c, 1});
613     }
614 }
615 }
616 sort(evt.begin(), evt.end());
617 if(u.a > u.b)
618     reverse(evt.begin(), evt.end());
619 int cnt = 0;
620 for(int i = 0; i < evt.size(); ++i)
621 {
622     cnt += evt[i].second;
623     const Point s = evt[i].first, t = evt[i + 1].first;
624     if(!(s == t) && check(s, u) && check(t, u))
625         segs[cnt].push_back({s, t});
626     if(t == u.b)
627         break;
628 }
629 };
630
631 for(int i = 0; i < polys.size(); ++i)
632 {
633     const auto & pi = polys[i];
634     for (int k = 0; k < pi.point.size(); ++k)
635     {
636         const Segment ei = {pi.point[k], pi.point[pi.nxt(k)]};
637         cut_edge(ei, i);
638     }
639 }
640 vector<T> ans(n);
641 for(int i = 0; i < n; ++i)
642 {
643     T sum = 0;
644     sort(segs[i].begin(), segs[i].end());
645     int cnt = 0;
646     for(int j = 0; j < segs[i].size(); ++j)
647     {
648         if(j > 0 && segs[i][j] == segs[i][j-1])
649             segs[i + (++cnt)].push_back(segs[i][j]);
650         else
651             cnt = 0, sum += segs[i][j].first ^ segs[i][j].second;
652     }
653     ans[i] = sum / 2;
654 }

```

```

655     return ans;
656 }
657
658 struct Circle
659 {
660     Point o;
661     T r;
662
663     Circle() {}
664     Circle(Point p, T _r) : o(p), r(_r) {}
665
666     bool operator == (const Circle & a) const
667     { return o == a.o && abs(r - a.r) <= eps; }
668     T circ() const {return 2 * PI * r;} // 周长
669     T area() const {return PI * r * r;} // 面积
670
671     // 点与圆的关系
672     // -1 圆上 | 0 圆外 | 1 圆内
673     int is_in(const Point & p) const
674     {
675         const T d = p.len2(o);
676         return abs(d - r * r) <= eps ? -1 : d < r * r - eps;
677     }
678
679     // 直线与圆关系
680     // 0 相离 | 1 相切 | 2 相交
681     int relation(const Line & l) const
682     {
683         const T d = l.dis(o);
684         if (d > r + eps)
685             return 0;
686         if (abs(d - r) <= eps)
687             return 1;
688         return 2;
689     }
690
691     // 圆与圆关系
692     // -1 相同 | 0 相离 | 1 外切 | 2 相交 | 3 内切 | 4 内含
693     int relation(const Circle & a) const
694     {
695         if(*this == a)
696             return -1;
697         const T d = o.len(a.o);
698         if(d > r + a.r + eps)
699             return 0;
700         if(abs(d - r - a.r) <= eps)
701             return 1;
702         if(abs(d - abs(r - a.r)) <= eps)
703             return 3;
704         if(d < abs(r - a.r) - eps)
705             return 4;
706         return 2;
707     }

```

```

708
709 // 直线与圆的交点
710 vector<Point> inter(const Line & l) const
711 {
712     const T d = l.dis(o);
713     const Point p = l.proj(o);
714     const int t = relation(l);
715     if (t == 0)
716         return vector<Point>();
717     if (t == 1)
718         return vector<Point>{p};
719     const T k = sqrt(r * r - d * d);
720     return vector<Point>{ p - (l.v / l.v.len()) * k, p + (l.v / l.v.len()) * k };
721 }
722
723 // 圆与圆交点
724 vector<Point> inter(const Circle & a) const
725 {
726     const T d = o.len(a.o);
727     const int t = relation(a);
728     if(t == -1 || t == 0 || t == 4)
729         return vector<Point>();
730     Point e = a.o - o;
731     e = e / e.len() * r;
732     if(t == 1 || t == 3)
733     {
734         if(r * r + d * d - a.r * a.r >= -eps)
735             return vector<Point>{o + e};
736         return vector<Point>{o - e};
737     }
738     const T costh = (r * r + d * d - a.r * a.r) / (2 * r * d), sinh = sqrt(1 -
costh * costh);
739     return vector<Point>{ o + e.rot(costh,-sinh), o + e.rot(costh,sinh) };
740 }
741
742 // 圆与圆交面积
743 T interArea(const Circle & a) const
744 {
745     const T d = o.len(a.o);
746     const int t = relation(a);
747     if(t == -1)
748         return area();
749     if(t < 2)
750         return 0;
751     if(t > 2)
752         return min(area(), a.area());
753     const T costh1 = (r * r + d * d - a.r * a.r) / (2 * r * d);
754     const T costh2 = (a.r * a.r + d * d - r * r) / (2 * a.r * d);
755     const T sinh1 = sqrt(1 - costh1 * costh1), sinh2 = sqrt(1 - costh2 *
costh2);
756     const T th1 = acos(costh1), th2 = acos(costh2);
757     return r * r * (th1 - costh1 * sinh1) + a.r * a.r * (th2 - costh2 * sinh2);
758 }

```

```

759 // 圆与多边形面积交
760 T interArea(const Polygon & plg)
761 {
762     const auto cal = [](const Circle & circ, const Point & a, const Point & b)
763     {
764         if((a - circ.o).topleft(b-circ.o) == 0)
765             return 0.0f;
766         const auto ina = circ.is_in(a), inb = circ.is_in(b);
767         const Line ab = {a, b - a};
768         if(ina && inb)
769             return ((a - circ.o) ^ (b - circ.o)) / 2;
770         if(ina && !inb)
771         {
772             const auto t = circ.inter(ab);
773             const Point p = t.size() == 1 ? t[0] : t[1];
774             const T ans = ((a - circ.o) ^ (p - circ.o)) / 2;
775             const T th = (p - circ.o).ang(b-circ.o);
776             const T d = circ.r * circ.r * th / 2;
777             if ((a - circ.o).topleft(b - circ.o) == 1)
778                 return ans + d;
779             return ans - d;
780         }
781         if(!ina && inb)
782         {
783             const Point p = circ.inter(ab)[0];
784             const T ans = ((p - circ.o) ^ (b - circ.o)) / 2;
785             const T th = (a - circ.o).ang(p - circ.o);
786             const T d = circ.r * circ.r * th / 2;
787             if((a - circ.o).topleft(b - circ.o) == 1)
788                 return ans+d;
789             return ans - d;
790         }
791         const auto p = circ.inter(ab);
792         if(p.size() == 2 && Segment(a, b).dis(circ.o) <= circ.r + eps)
793         {
794             const T ans = ((p[0]-circ.o) ^ (p[1] - circ.o)) / 2;
795             const T th1 = (a - circ.o).ang(p[0] - circ.o), th2 = (b -
796             circ.o).ang(p[1] - circ.o);
797             const T d1 = circ.r * circ.r * th1 / 2, d2 = circ.r * circ.r * th2 /
798             2;
799             if ((a - circ.o).topleft(b - circ.o) == 1)
800                 return ans + d1 + d2;
801             return ans - d1 - d2;
802         }
803         const T th = (a - circ.o).ang(b - circ.o);
804         if((a - circ.o).topleft(b - circ.o) == 1)
805             return circ.r * circ.r * th / 2;
806         return -circ.r * circ.r * th / 2;
807     };
808     T ans = 0;
809     for (int i = 0; i < plg.point.size(); ++i)

```



```

810     {
811         const Point a = plg.point[i], b = plg.point[plg.nxt(i)];
812         ans += cal(*this, a, b);
813     }
814     return ans;
815 }
816
817 // 过圆外一点圆的切线
818 vector<Line> tangent(const Point & a) const
819 {
820     const int t = is_in(a);
821     if(t == 1)
822         return vector<Line>();
823     if(t == -1)
824     {
825         const Point v = {-(a - o).y, (a - o).x};
826         return vector<Line>{{a, v}};
827     }
828     Point e = a - o;
829     e = e / e.len() * r;
830     const T costh = r / o.len(a), sinh = sqrt(1 - costh * costh);
831     const Point t1 = o + e.rot(costh, -sinh), t2 = o + e.rot(costh, sinh);
832     return vector<Line>{{a, t1 - a}, {a, t2 - a}};
833 }
834
835 // 两圆的公切线
836 vector<Line> tangent(const Circle & a) const
837 {
838     const int t = relation(a);
839     vector<Line> lines;
840     if (t == -1 || t == 4)
841         return lines;
842     // t=1 (外切): 这条线是过切点的"内公切线", 垂直于圆心连线。
843     //             它把两个圆隔开, 不在凸包边界上。-> 必须去掉
844     // t=3 (内切): 虽然是外围的线, 但在圆凸包算法中, 被包含的圆通常不贡献边界。
845     //             且该点容易导致数值不稳定。-> 去掉通常无影响
846     if (t == 1 || t == 3)
847     {
848         const Point p = inter(a)[0], v = {-(a.o - o).y, (a.o - o).x};
849         lines.push_back({p, v});
850     }
851     const T d = o.len(a.o);
852     const Point e = (a.o - o) / (a.o - o).len();
853     if (t <= 2) // 外公切线
854     {
855         const T costh = (r - a.r) / d, sinh = sqrt(1 - costh * costh);
856         const Point d1 = e.rot(costh, -sinh), d2 = e.rot(costh, sinh);
857         const Point u1 = o + d1 * r, u2 = o + d2 * r, v1 = a.o + d1 * a.r, v2 =
a.o + d2 * a.r;
858         lines.push_back({u1, v1 - u1}), lines.push_back({u2, v2 - u2});
859     }
860     if (t == 0) // 内公切线
861     {

```

```

862         const T costh = (r + a.r) / d, sinth = sqrt(1 - costh * costh);
863         const Point d1 = e.rot(costh, -sinth), d2 = e.rot(costh, sinth);
864         const Point u1 = o + d1 * r, u2 = o + d2 * r, v1 = a.o - d1 * a.r, v2 =
a.o - d2 * a.r;
865         lines.push_back({u1, v1 - u1}), lines.push_back({u2, v2 - u2});
866     }
867     return lines;
868 }
869
870 //圆的反演
871 // auto result = circle.inverse(line);
872 // if (std::holds_alternative<Circle>(result))
873 // Circle c = std::get<Circle>(result);
874 variant<Circle, Line> inverse(const Line & L) const
875 {
876     if(L.toleft(o)==0)
877         return L;
878     const Point v = L.toleft(o) == 1 ? Point(L.v.y, -L.v.x) : Point(-L.v.y,
L.v.x);
879     const T d = r * r / L.dis(o);
880     const Point p = o + v / v.len() * d;
881     return Circle((o + p) / 2, d / 2);
882 }
883 variant<Circle, Line> inverse(const Circle & a) const
884 {
885     const Point v = a.o - o;
886     if(a.is_in(o) == -1)
887     {
888         const T d = r * r / (a.r + a.r);
889         const Point p = o + v / v.len() * d;
890         return Line(p, Point(-v.y,v.x));
891     }
892     if(o == a.o)
893         return Circle(o, r * r / a.r);
894     const T d1 = r * r / (o.len(a.o) - a.r), d2 = r * r / (o.len(a.o) + a.r);
895     const Point p = o + v / v.len() * d1, q = o + v / v.len() * d2;
896     return Circle((p + q) / 2, p.len(q) / 2);
897 }
898 };
899
900 // 给定三个点求外接圆，注意自定义共线情况
901 Circle CircumCircle(Point A, Point B, Point C)
902 {
903     Line L1((A + B) / 2, (B - A).rot90());
904     Line L2((A + C) / 2, (C - A).rot90());
905     if(!L1.isConnect(L2))
906     {
907         // 三点共线，理论上不应进入此分支，除非数据有误或逻辑需要退化处理
908         // 在SEC算法中，这里可以 assert(0) 或者返回两点直径圆
909         // 值得注意的是如果有点重合出现零向量方向向量的直线也会判成平行
910         // assert(1);
911         return {{0, 0}, INF};
912     }

```

```

913     Point center = L1.inter(L2);
914     return Circle(center, center.len(A));
915 }
916
917 // 给定三个点求内切圆
918 Circle InCircle(Point A, Point B, Point C)
919 {
920     T a = B.len(C), b = A.len(C), c = A.len(B);
921     T area = abs((B - A) ^ (C - A));
922     T sum = a + b + c;
923     Point center = (A * a + B * b + C * c) / (a + b + c);
924     T r = area / (a + b + c);
925     return Circle(center, r);
926 }
927
928 // 最小圆覆盖 最坏 $O(n^3)$ , 随机化下认为是 $O(n)$ 
929 // https://ac.nowcoder.com/acm/contest/52826/D
930 // https://www.luogu.com.cn/problem/A742
931 Circle min_cover_circle(vector<Point> ps)
932 {
933     int n = ps.size();
934     random_shuffle(ps.begin(), ps.end());
935     Circle cir(ps[0], 0);
936     for(int i = 0; i < n; i++)
937     {
938         if(cir.is_in(ps[i]))
939             continue;
940         cir.o = ps[i], cir.r = 0;
941         for(int j = 0; j < i; j++)
942         {
943             if(cir.is_in(ps[j]))
944                 continue;
945             cir.o = (ps[i] + ps[j]) / 2;
946             cir.r = ps[i].len(ps[j]) / 2;
947             for (int k = 0; k < j; k++)
948             {
949                 if (cir.is_in(ps[k]))
950                     continue;
951                 cir = CircumCircle(ps[i], ps[j], ps[k]);
952             }
953         }
954     }
955     return cir;
956 }
957
958 // 构建阿波罗尼斯圆: 集合 P 满足  $\text{dist}(P, B) \leq k * \text{dist}(P, A)$  的边界圆
959 // k: 比率 (必须  $k \neq 1$ , 且  $k > 0$ )
960 // 如果  $k < 1$ , 圆包裹点 B。
961 // 如果  $k > 1$ , 圆包裹点 A。
962 // 如果  $k = 1$ , 这是垂直平分线
963 Circle getApolloniusCircle(Point A, Point B, T k)
964 {
965     assert(abs(k - 1.0) > eps && k > eps); // 处理特殊情况 k=1 (垂直平分线, 不是圆)

```

```

966     T k2 = k * k;
967     Point o = (B - A * k2) / (1.0 - k2);
968     T dist_AB = (A - B).len();
969     T r = (k * dist_AB) / abs(1.0 - k2);
970     return {o, r};
971 }
972
973 // 圆面积并
974 // 轮廓积分, 复杂度  $O(n^2 \log n)$ 
975 // ans[i] 表示被至少覆盖了 i+1 次的区域的面积, 再进行一个容斥可得恰好 i+1 次
976 // https://ac.nowcoder.com/acm/contest/40275/G
977 vector<T> area_union(const vector<Circle> & circs)
978 {
979     const size_t siz = circs.size();
980     using arc_t = tuple<Point, T, T, T>;
981     vector<vector<arc_t>> arcs(siz);
982     const auto eq = [](const arc_t & u, const arc_t & v)
983     {
984         const auto [u1,u2,u3,u4] = u;
985         const auto [v1,v2,v3,v4] = v;
986         return u1 == v1 && abs(u2 - v2) <= eps && abs(u3 - v3) <= eps && abs(u4 - v4)
987         <= eps;
988     };
989     auto cut_circ=[&](const Circle & ci, const size_t i)
990     {
991         vector<pair<T, int>> evt;
992         evt.push_back({-PI, 0}), evt.push_back({PI, 0});
993         int init = 0;
994         for(size_t j = 0; j < circs.size(); ++j)
995         {
996             if(i == j)
997                 continue;
998             const Circle & cj = circs[j];
999             if(ci.r < cj.r - eps && ci.relation(cj) >= 3)
1000                 init++;
1001             const auto inters = ci.inter(cj);
1002             if(inters.size() == 1)
1003                 evt.push_back({atan2l((inters[0] - ci.o).y, (inters[0] - ci.o).x),
1004 0});
1005             if(inters.size() == 2)
1006             {
1007                 const Point d1 = inters[0] - ci.o, dr = inters[1] - ci.o;
1008                 T arg1 = atan2l(d1.y, d1.x), argr = atan2l(dr.y, dr.x);
1009                 if(abs(arg1 + PI) <= eps)
1010                     arg1=PI;
1011                 if(abs(argr + PI) <= eps)
1012                     argr=PI;
1013                 if(arg1 > argr + eps)
1014                 {
1015                     evt.push_back({arg1, 1});
1016                     evt.push_back({PI, -1});
1017                     evt.push_back({-PI, 1});

```

```

1017         evt.push_back({argr, -1});
1018     }
1019     else
1020     {
1021         evt.push_back({argl, 1});
1022         evt.push_back({argr, -1});
1023     }
1024 }
1025 }
1026 sort(evt.begin(), evt.end());
1027 int sum = init;
1028 for(size_t i = 0; i < evt.size(); ++i)
1029 {
1030     sum += evt[i].second;
1031     if(abs(evt[i].first - evt[i+1].first) > eps)
1032         arcs[sum].push_back({ci.o, ci.r, evt[i].first, evt[i+1].first});
1033     if(abs(evt[i+1].first-PI)<=eps)
1034         break;
1035 }
1036 };
1037 const auto oint = [](const arc_t & arc)
1038 {
1039     const auto [cc,cr,l,r] = arc;
1040     if(abs(r - l - PI - PI) <= eps)
1041         return 2.01 * PI * cr * cr;
1042     return cr * cr * (r - l) + cc.x * cr * (sin(r) - sin(l)) - cc.y * cr *
(cos(r) - cos(l));
1043 };
1044 for(size_t i = 0; i < circs.size(); ++i)
1045 {
1046     const auto &ci = circs[i];
1047     cut_circ(ci, i);
1048 }
1049 vector<T> ans(siz);
1050 for(size_t i = 0; i < siz; ++i)
1051 {
1052     T sum = 0;
1053     sort(arcs[i].begin(), arcs[i].end());
1054     int cnt = 0;
1055     for(size_t j = 0; j < arcs[i].size(); ++j)
1056     {
1057         if(j > 0 && eq(arcs[i][j], arcs[i][j-1]))
1058             arcs[i + (++cnt)].push_back(arcs[i][j]);
1059         else
1060             cnt = 0, sum += oint(arcs[i][j]);
1061     }
1062     ans[i]=sum / 2;
1063 }
1064 return ans;
1065 }
1066
1067 // ---- 临时 ----
1068

```

```

1069 // 三角形的三个下标; 约定 a,b,c 始终为逆时针。
1070 struct Triangle
1071 {
1072     int a = 0, b = 0, c = 0;
1073     Triangle(int _a = 0, int _b = 0, int _c = 0) : a(_a), b(_b), c(_c) {}
1074 };
1075
1076 T triangle_area2(const Point &a, const Point &b, const Point &c)
1077 {
1078     return (b - a) ^ (c - a);
1079 }
1080
1081 bool in_triangle_closed(const Point &p, const Point &a, const Point &b, const Point
&c)
1082 {
1083     // a,b,c 为逆时针; 边界也算在三角形内。
1084     return triangle_area2(a, b, p) >= -eps
1085         && triangle_area2(b, c, p) >= -eps
1086         && triangle_area2(c, a, p) >= -eps;
1087 }
1088
1089 // 多边形三角剖分
1090 vector<Triangle> polygon_triangulation(const Polygon &plg)
1091 {
1092     vector<Triangle> ret;
1093     const int n = plg.point.size();
1094     vector<int> v;
1095     for(int i = 0; i < n; ++i)
1096         v.push_back(i);
1097     if(v.size() < 3)
1098         return ret;
1099     auto area_of = [&](const vector<int> &ids)
1100     {
1101         T ans = 0;
1102         for (int i = 0; i < ids.size(); ++i)
1103             ans += plg.point[ids[i]] ^ plg.point[ids[(i + 1) % ids.size()]];
1104         return ans;
1105     };
1106     T area = area_of(v);
1107     if(abs(area) <= eps)
1108         return ret;
1109     const int m = v.size();
1110     if(m < 3 || abs(area_of(v)) <= eps)
1111         return ret;
1112     vector<int> pre(m), nxt(m);
1113     vector<char> alive(m, true);
1114     iota(pre.begin(), pre.end(), 0);
1115     iota(nxt.begin(), nxt.end(), 0);
1116     for(int i = 0; i < m; ++i)
1117     {
1118         pre[i] = (i + m - 1) % m;
1119         nxt[i] = (i + 1) % m;
1120     }

```

```

1121     auto is_ear = [&](int i)
1122     {
1123         if (!alive[i])
1124             return false;
1125         const int l = pre[i], r = nxt[i];
1126         const Point &a = plg.point[v[l]];
1127         const Point &b = plg.point[v[i]];
1128         const Point &c = plg.point[v[r]];
1129         if(triangle_area2(a, b, c) <= eps)
1130             return false;
1131         for(int j = 0; j < m; ++j)
1132         {
1133             if (!alive[j] || j == l || j == i || j == r)
1134                 continue;
1135             if (in_triangle_closed(plg.point[v[j]], a, b, c))
1136                 return false;
1137         }
1138         return true;
1139     };
1140     set<int> ears;
1141     for (int i = 0; i < m; ++i)
1142         if (is_ear(i))
1143             ears.insert(i);
1144     int left = m;
1145     while(left > 3)
1146     {
1147         if (ears.empty())
1148             return vector<Triangle>(); // 自交或数值退化等非法输入
1149         const int i = *ears.begin();
1150         ears.erase(ears.begin());
1151         if (!alive[i] || !is_ear(i))
1152             continue; // 邻点更新后可能留下过期标记
1153         const int l = pre[i], r = nxt[i];
1154         ret.push_back({v[l], v[i], v[r]});
1155         alive[i] = false;
1156         nxt[l] = r;
1157         pre[r] = l;
1158         --left;
1159         ears.erase(l);
1160         ears.erase(r);
1161         if (is_ear(l))
1162             ears.insert(l);
1163         if (is_ear(r))
1164             ears.insert(r);
1165     }
1166     int a = -1, b = -1, c = -1;
1167     for (int i = 0; i < m; ++i)
1168         if (alive[i])
1169         {
1170             if (a == -1) a = i;
1171             else if (b == -1) b = i;
1172             else c = i;
1173         }

```

```

1174     if (a == -1 || triangle_area2(plg.point[v[a]], plg.point[v[b]], plg.point[v[c]])
    <= eps)
1175         return vector<Triangle>();
1176     ret.push_back({v[a], v[b], v[c]});
1177     if (ret.size() != m - 2)
1178         return vector<Triangle>();
1179     return ret;
1180 }

```

三维几何

```

1  #pragma once
2  // -----
3  // 3D Geometry Competitive Template (Styled & Documented)
4  // -----
5  // 保留竞赛习惯宏: #define int long long 与 #define T long double
6  // - 目标: 一份可直接在算法竞赛中粘贴使用的 3D 几何“万金油”板子
7  // - 风格: 统一空格与缩进, 二元运算符两侧留空格 (如 a - b, x * y)
8  // - 数值: 默认使用 long double, 所有三角/开方均做 NaN 防护
9  // - 覆盖: 点/线/面/球/圆/三角形、射线/线段相交、截面重建、平面凸包、应急 3D 凸包等
10 // - 注释: 每个函数给出用途、参数、返回值、边界/退化与复杂度 (如有)
11 // -----
12 // 依赖: <bits/stdc++.h> 可在你的主文件中包含; 此处仅作为头文件使用。
13 // -----
14
15 #include <cmath>
16 #include <vector>
17 #include <array>
18 #include <algorithm>
19 #include <numeric>
20 #include <unordered_map>
21
22 using std::array;
23 using std::pair;
24 using std::vector;
25 using std::size_t;
26
27 // ===== 竞赛宏 (按你的习惯保留) =====
28 #define int long long
29 #define T long double
30
31 // ===== 常量与数值工具 =====
32 static const T EPS = 1e-12L; // 主 EPS: 针对 long double, 若数据量级很大可相对化处理
33 static const T PI = acos(-1.0L);
34
35 inline int sgn(T x) {
36     // 三态比较: >EPS 返回 1, <-EPS 返回 -1, 否则 0
37     if (x > EPS) return 1;

```



```

38     if (x < -EPS) return -1;
39     return 0;
40 }
41
42 inline int dcmp(T x, T y) {
43     // 自适应比较: 允许相对/绝对误差
44     const T diff = x - y;
45     if (fabsl(diff) <= EPS * std::max<T>(1.0L, std::max(fabsl(x), fabsl(y)))) return
0;
46     return diff < 0 ? -1 : 1;
47 }
48
49 inline T clamp_pm1(T v) {
50     // 将值夹到 [-1, 1], 避免 acos/asin 出现 NaN
51     if (v < -1.0L) return -1.0L;
52     if (v > 1.0L) return 1.0L;
53     return v;
54 }
55
56 inline T safe_sqrt(T x) {
57     // 对 -1e-18 这类“负零”先截断为 0 再开方
58     return x <= 0 ? 0 : sqrtl(x);
59 }
60
61 inline T clampT(T x, T lo, T hi) {
62     // 将 x 夹到 [lo, hi]
63     if (x < lo) return lo;
64     if (x > hi) return hi;
65     return x;
66 }
67
68 // ===== 基本结构: 点 / 直线 / 线段 / 射线 / 平面 =====
69 struct Point3 {
70     T x { 0 }, y { 0 }, z { 0 };
71
72     // --- 算子: 保持两侧空格, 易读 ---
73     bool operator == (const Point3 & a) const {
74         return fabsl(x - a.x) <= EPS && fabsl(y - a.y) <= EPS && fabsl(z - a.z) <=
EPS;
75     }
76     Point3 operator + (const Point3 & a) const { return { x + a.x, y + a.y, z + a.z
}; }
77     Point3 operator - (const Point3 & a) const { return { x - a.x, y - a.y, z - a.z
}; }
78     Point3 operator - () const { return { -x, -y, -z }; }
79     Point3 operator * (T k) const { return { x * k, y * k, z * k }; }
80     Point3 operator / (T k) const { return { x / k, y / k, z / k }; }
81
82     // 点积: 按你的习惯重载 *
83     T operator * (const Point3 & a) const { return x * a.x + y * a.y + z * a.z; }
84
85     // 范数、距离
86     T len2() const { return (*this) * (*this); }

```

```

87     T len() const { return sqrt1(len2()); }
88     T dis2(const Point3 & a) const { return (a - *this).len2(); }
89     T dis (const Point3 & a) const { return sqrt1(dis2(a)); }
90 };
91 inline Point3 operator * (T k, const Point3 & p) { return p * k; }
92 inline T      Dot(const Point3 & a, const Point3 & b) { return a * b; }
93 inline Point3 Cross(const Point3 & a, const Point3 & b) {
94     return { a.y * b.z - a.z * b.y,
95             a.z * b.x - a.x * b.z,
96             a.x * b.y - a.y * b.x };
97 }
98
99 inline T Angle(const Point3 & a, const Point3 & b) {
100     // 向量夹角 (返回  $[0, \pi]$ )。若含零向量, 按 0 处理。
101     const T la = a.len(), lb = b.len();
102     if (la <= EPS || lb <= EPS) return 0;
103     const T c = clamp_pm1((a * b) / (la * lb));
104     return acos1(c);
105 }
106
107 struct Line3 { Point3 p1, p2; };           // 无限直线: 过 p1, p2 (p1 != p2)
108 using Segment3 = Line3;                 // 线段: 同结构, 参数 t ∈ [0, 1]
109 struct Ray3 { Point3 o, d; };            // 射线: o + t * d, t >= 0
110
111 inline Point3 dir(const Line3 & L) { return L.p2 - L.p1; }
112
113 inline bool point_on_line(const Point3 & p, const Line3 & L) {
114     // 点在直线上 (无限延长): (p1 - p) × (p2 - p) 的长度为 0
115     const T num = Cross(L.p1 - p, L.p2 - p).len();
116     const T den = (L.p1 - p).len() * (L.p2 - p).len();
117     return safe_sqrt(num * num) <= EPS * den;
118 }
119
120 inline bool point_on_segment(const Point3 & p, const Segment3 & S) {
121     // 点在线段上: 共线 + 投影参数在 [0,1]
122     if (!point_on_line(p, S)) return false;
123     return sgn((p - S.p1) * (S.p2 - S.p1)) >= 0 && sgn((p - S.p2) * (S.p1 - S.p2)) >=
0;
124 }
125
126 inline T Dis_point_line(Point3 p, Line3 L) {
127     // 点到直线距离: | (p - p1) × (p2 - p1) | / |p2 - p1|
128     return Cross(L.p2 - L.p1, p - L.p1).len() / L.p1.dis(L.p2);
129 }
130
131 inline Point3 Point_line_proj(Point3 p, Line3 L) {
132     // 点在直线上的投影: p1 + d * ( (p - p1)·d / |d|^2 )
133     const Point3 d = L.p2 - L.p1;
134     const T k = (d * (p - L.p1)) / d.len2();
135     return L.p1 + d * k;
136 }
137
138 inline T Dis_point_seg(Point3 p, Segment3 S) {

```

```

139 // 点到线段最短距离: 投影若不在段内, 取端点最小距离
140 if (sgn((p - s.p1) * (S.p2 - s.p1)) < 0 || sgn((p - s.p2) * (s.p1 - s.p2)) < 0)
141     return std::min(p.dis(s.p1), p.dis(s.p2));
142 return Dis_point_line(p, S);
143 }
144
145 struct Plane {
146     // 用三点定义平面 (需不共线)。
147     Point3 p1, p2, p3;
148
149     bool valid() const { return Cross(p2 - p1, p3 - p1).len() > EPS; }
150
151     Point3 unit_normal() const {
152         // 返回单位法向量 (调用前需 valid())
153         const Point3 n = Cross(p2 - p1, p3 - p1);
154         return n / n.len();
155     }
156
157     T signed_distance(const Point3 & p) const {
158         // 有符号距离 (朝向由法向决定)
159         const Point3 n = unit_normal();
160         return n * (p - p1);
161     }
162
163     T distance(const Point3 & p) const {
164         // 绝对距离
165         return fabs1(signed_distance(p));
166     }
167
168     Point3 project(const Point3 & p) const {
169         // p 在该平面上的正交投影
170         const Point3 n = unit_normal();
171         return p - n * (n * (p - p1));
172     }
173
174     // 兼容旧接口: get_dis == signed_distance
175     T get_dis(const Point3 & p) const { return signed_distance(p); }
176 };
177
178 inline Point3 pvec(Point3 A, Point3 B, Point3 C) { return Cross(B - A, C - A); }
179 inline Point3 pvec(Plane f) { return Cross(f.p2 - f.p1, f.p3 - f.p1); }
180
181 inline bool coplanar(Point3 A, Point3 B, Point3 C, Point3 D) {
182     // 四点共面:  $(D - A) \cdot ((B - A) \times (C - A)) == 0$ 
183     return sgn(pvec(A, B, C) * (D - A)) == 0;
184 }
185
186 inline bool parallel(Plane f1, Plane f2) {
187     // 两平面平行: 法向叉积为 0
188     return Cross(pvec(f1), pvec(f2)).len() < EPS;
189 }
190
191 inline bool vertical(Plane f1, Plane f2) {

```

```

192 // 两平面垂直：法向量积为 0
193 return sgn(pvec(f1) * pvec(f2)) == 0;
194 }
195
196 inline bool plane_from_point_normal(const Point3 & p, Point3 n, Plane & out) {
197 // 通过点 p 与法向 n (可非单位) 构造平面；返回是否成功 (n 非零)
198 if (n.len() <= EPS) return false;
199 Point3 a = fabs1(n.x) > fabs1(n.y) ? Point3{ -n.z, 0, n.x } : Point3{ 0, n.z, -
n.y };
200 if (a.len() <= EPS) a = { 1, 0, 0 };
201 const Point3 u = Cross(n, a);
202 if (u.len() <= EPS) return false;
203 out = Plane{ p, p + u, p + Cross(n, u) };
204 return out.valid();
205 }
206
207 inline int Line_cross_plane(Line3 u, Plane f, Point3 & p) {
208 // 直线与平面相交：返回 -1 共面；0 平行不交；1 唯一交点 (写入 p)
209 if (!f.valid()) return 0;
210 const Point3 n = pvec(f);
211 const Point3 d = u.p2 - u.p1;
212 const T denom = n * d;
213 if (sgn(denom) == 0) {
214     if (sgn(n * (u.p1 - f.p1)) == 0) return -1;
215     return 0;
216 }
217 const T t = (n * (f.p1 - u.p1)) / denom;
218 p = u.p1 + d * t;
219 return 1;
220 }
221
222 // ===== 两直线/三直线确定平面 & 线线关系 =====
223 inline bool coplanar_lines(const Line3 & L1, const Line3 & L2) {
224 // 两直线共面：(P2 - P1) · (d1 × d2) == 0
225 const Point3 d1 = dir(L1);
226 const Point3 d2 = dir(L2);
227 return sgn(Dot(L2.p1 - L1.p1, Cross(d1, d2))) == 0;
228 }
229
230 inline int plane_from_two_lines(const Line3 & A, const Line3 & B, Plane & out) {
231 // 两直线确定平面：
232 // 返回 1 唯一；0 异面；-1 重合；-2 退化
233 if (A.p1 == A.p2) return -2;
234 if (B.p1 == B.p2) return -2;
235 const Point3 d1 = dir(A);
236 const Point3 d2 = dir(B);
237 const Point3 c = Cross(d1, d2);
238 if (c.len() > EPS) {
239     if (!coplanar_lines(A, B)) return 0;
240     out = Plane{ A.p1, A.p1 + d1, A.p1 + d2 };
241     return 1;
242 } else {
243     const Point3 v = B.p1 - A.p1;

```

```

244         if (Cross(v, d1).len() <= EPS) return -1; // 重合
245         out = Plane{ A.p1, A.p1 + d1, B.p1 }; // 平行不重合 → 唯一
246         return 1;
247     }
248 }
249
250 inline int plane_from_three_lines(const Line3 & L1, const Line3 & L2, const Line3 &
L3, Plane & out) {
251     // 三直线确定平面:
252     // 返回 1 唯一; 0 无; -1 无穷; -2 退化
253     auto in_plane = [&](const Line3 & L, const Plane & P) { return P.distance(L.p1)
<= EPS && P.distance(L.p2) <= EPS; };
254     Plane tmp; int r;
255     r = plane_from_two_lines(L1, L2, tmp);
256     if (r == 1) { if (in_plane(L3, tmp)) { out = tmp; return 1; } else return 0; } if
(r == -2) return -2;
257     r = plane_from_two_lines(L1, L3, tmp);
258     if (r == 1) { if (in_plane(L2, tmp)) { out = tmp; return 1; } else return 0; } if
(r == -2) return -2;
259     r = plane_from_two_lines(L2, L3, tmp);
260     if (r == 1) { if (in_plane(L1, tmp)) { out = tmp; return 1; } else return 0; } if
(r == -2) return -2;
261     if (r == -1) return -1;
262     return 0;
263 }
264
265 inline int closest_points_on_lines(const Line3 & L1, const Line3 & L2, Point3 & A,
Point3 & B) {
266     // 两直线最近点对（无限直线）。
267     // 返回 1 正常; -1 某一条退化为点; -2 两条都退化为点。
268     const Point3 p = L1.p1, q = L2.p1;
269     const Point3 u = dir(L1), v = dir(L2), w = p - q;
270     const T a = Dot(u, u), b = Dot(u, v), c = Dot(v, v), d = Dot(u, w), e = Dot(v,
w);
271     if (a <= EPS && c <= EPS) { A = p; B = q; return -2; }
272     if (a <= EPS) { const T s = e / c; A = p; B = q + v * s; return -1; }
273     if (c <= EPS) { const T t = -d / a; A = p + u * t; B = q; return -1; }
274     const T D = a * c - b * b;
275     T sc, tc;
276     if (fabs1(D) <= EPS) { sc = 0; tc = e / c; }
277     else { sc = (b * e - c * d) / D; tc = (a * e - b * d) / D; }
278     A = p + u * sc; B = q + v * tc; return 1;
279 }
280
281 inline T line_line_distance(const Line3 & L1, const Line3 & L2) {
282     // 两直线最短距离
283     Point3 A, B; (void)closest_points_on_lines(L1, L2, A, B);
284     return (A - B).len();
285 }
286
287 inline int line_line_intersection(const Line3 & L1, const Line3 & L2, Point3 & p) {
288     // 两直线相交测试（无限直线）。
289     // 返回 1 唯一交点; 0 不相交; -1 重合

```

```

290     if (!coplanar_lines(L1, L2)) return 0;
291     const Point3 u = dir(L1), v = dir(L2);
292     if (Cross(u, v).len() <= EPS) {
293         if (Cross(L2.p1 - L1.p1, u).len() <= EPS) return -1; // 重合
294         return 0; // 平行不重合
295     }
296     Point3 A, B; (void)closest_points_on_lines(L1, L2, A, B);
297     if ((A - B).len() <= EPS) { p = A; return 1; }
298     return 0;
299 }
300
301 inline T clamp01(T t) { return t < 0 ? 0 : (t > 1 ? 1 : t); }
302
303 inline T segment_segment_distance(const Segment3 & S1, const Segment3 & S2, Point3 &
SA, Point3 & SB) {
304     // 两线段间最短距离, 同时返回最近点 SA/SB。
305     const Point3 p = S1.p1, q = S2.p1;
306     const Point3 u = S1.p2 - S1.p1, v = S2.p2 - S2.p1, w = p - q;
307     const T a = Dot(u, u), b = Dot(u, v), c = Dot(v, v), d = Dot(u, w), e = Dot(v,
w);
308     const T D = a * c - b * b;
309     T sN, sD = D, tN, tD = D;
310     if (a <= EPS && c <= EPS) { SA = p; SB = q; return (SA - SB).len(); }
311     if (a <= EPS) { sN = 0; sD = 1; tN = e; tD = c; }
312     else if (c <= EPS) { tN = 0; tD = 1; sN = -d; sD = a; }
313     else {
314         sN = (b * e - c * d);
315         tN = (a * e - b * d);
316         if (sN < 0) { sN = 0; tN = e; tD = c; }
317         else if (sN > sD) { sN = sD; tN = e + b; tD = c; }
318     }
319     if (tN < 0) { tN = 0; if (-d < 0) sN = 0; else if (-d > a) sN = sD; else { sN = -
d; sD = a; } }
320     else if (tN > tD) { tN = tD; if ((-d + b) < 0) sN = 0; else if ((-d + b) > a) sN
= sD; else { sN = (-d + b); sD = a; } }
321     const T sc = (fabs1(sN) <= EPS ? 0 : sN / sD);
322     const T tc = (fabs1(tN) <= EPS ? 0 : tN / tD);
323     SA = p + u * sc; SB = q + v * tc;
324     return (SA - SB).len();
325 }
326
327 // ===== 平面-平面交线 & 夹角 =====
328 inline int plane_plane_intersection(const Plane & P1, const Plane & P2, Line3 & L) {
329     // 两平面交: 返回 1 唯一交线; 0 平行不同面; -1 重合
330     if (!P1.valid() || !P2.valid()) return 0;
331     const Point3 n1 = pvec(P1), n2 = pvec(P2), d = Cross(n1, n2);
332     if (d.len() <= EPS) {
333         if (sgn(Dot(n1, P2.p1 - P1.p1)) == 0) return -1;
334         return 0;
335     }
336     const T c1 = Dot(n1, P1.p1), c2 = Dot(n2, P2.p1);
337     const Point3 n2xd = Cross(n2, d), dxn1 = Cross(d, n1);
338     const T denom = Dot(n1, n2xd);

```

```

339     const Point3 point = (c1 * n2xd + c2 * dxn1) / denom;
340     L = Line3{ point, point + d };
341     return 1;
342 }
343
344 inline T angle_between_lines(const Line3 & L1, const Line3 & L2) {
345     // 直线与直线夹角 ( $[0, \pi]$ )
346     const Point3 u = dir(L1), v = dir(L2);
347     const T c = clamp_pm1(Dot(u, v) / (u.len() * v.len()));
348     return acos1(c);
349 }
350
351 inline T angle_line_plane(const Line3 & L, const Plane & P) {
352     // 直线与平面夹角 (线与法向的余角): 返回  $[0, \pi/2]$ 
353     const Point3 u = dir(L), n = P.unit_normal();
354     T s = fabs1(Dot(u, n)) / (u.len()); s = clamp_pm1(s);
355     return asin1(s);
356 }
357
358 inline T angle_between_planes(const Plane & P1, const Plane & P2) {
359     // 平面与平面夹角 (取锐角): 返回  $[0, \pi/2]$ 
360     const Point3 n1 = P1.unit_normal(), n2 = P2.unit_normal();
361     const T c = clamp_pm1(fabs1(Dot(n1, n2)));
362     return acos1(c);
363 }
364
365 // ===== AABB 快速判交 =====
366 struct AABB { Point3 mn, mx; };
367
368 inline bool aabb_valid(const AABB & b) {
369     return b.mn.x <= b.mx.x && b.mn.y <= b.mx.y && b.mn.z <= b.mx.z;
370 }
371
372 inline bool segment_aabb_intersect(const Segment3 & S, const AABB & B) {
373     // Slab 法: 线段与 AABB 判交 (返回是否相交)
374     const Point3 d = S.p2 - S.p1; // 参数  $t \in [0, 1]$ 
375     T tmin = 0, tmax = 1;
376
377     auto upd = [&](T p, T q) {
378         //  $p = d[i]$ ,  $q = (\text{bound} - S.p1[i])$ ;  $p \approx 0$  视为平行, 由其它维度约束
379         if (fabs1(p) <= EPS) return true;
380         const T t = q / p;
381         if (p > 0) tmin = std::max(tmin, t); else tmax = std::min(tmax, t);
382         return true;
383     };
384
385     for (int k = 0; k < 3; ++k) {
386         const T Sp = (k == 0 ? S.p1.x : (k == 1 ? S.p1.y : S.p1.z));
387         const T Sd = (k == 0 ? d.x : (k == 1 ? d.y : d.z));
388         const T L = (k == 0 ? B.mn.x : (k == 1 ? B.mn.y : B.mn.z));
389         const T R = (k == 0 ? B.mx.x : (k == 1 ? B.mx.y : B.mx.z));
390         upd(Sd, L - Sp);
391         upd(-Sd, Sp - R);

```

```

392     }
393     return tmin <= tmax + EPS;
394 }
395
396 inline bool ray_aabb_intersect(const Point3 & o, const Point3 & d, const AABB & B, T
& t_hit) {
397     // 射线与 AABB: 返回是否命中, 并给出最近 t >= 0
398     T t1 = -1e100L, t2 = 1e100L;
399     auto slab = [&](T oo, T dd, T mn, T mx) -> pair<T, T> {
400         if (fabs1(dd) <= EPS) return (oo >= mn - EPS && oo <= mx + EPS) ? pair<T, T>{
-1e100L, 1e100L } : pair<T, T>{ 1, 0 };
401         T a = (mn - oo) / dd, b = (mx - oo) / dd; if (a > b) std::swap(a, b); return
{ a, b };
402     };
403     auto sx = slab(o.x, d.x, B.mn.x, B.mx.x); t1 = std::max(t1, sx.first); t2 =
std::min(t2, sx.second);
404     auto sy = slab(o.y, d.y, B.mn.y, B.mx.y); t1 = std::max(t1, sy.first); t2 =
std::min(t2, sy.second);
405     auto sz = slab(o.z, d.z, B.mn.z, B.mx.z); t1 = std::max(t1, sz.first); t2 =
std::min(t2, sz.second);
406     if (t1 > t2 + EPS || t2 < 0) return false;
407     t_hit = (t1 >= 0 ? t1 : t2);
408     return true;
409 }
410
411 struct Sphere { Point3 c; T r; };
412
413 inline bool sphere_aabb_intersect(const Sphere & S, const AABB & B) {
414     // 球与 AABB: 最近点到盒子距离 ≤ r 则相交
415     const Point3 q = { clampT(S.c.x, B.mn.x, B.mx.x),
416                       clampT(S.c.y, B.mn.y, B.mx.y),
417                       clampT(S.c.z, B.mn.z, B.mx.z) };
418     return (q - S.c).len2() <= S.r * S.r + EPS;
419 }
420
421 // ===== 三平面交点 =====
422 inline bool plane_plane_plane_intersection(const Plane & P1, const Plane & P2, const
Plane & P3, Point3 & X) {
423     // 三个一般位置平面的唯一交点 (若存在)
424     const Point3 n1 = pvec(P1), n2 = pvec(P2), n3 = pvec(P3);
425     const T d1 = Dot(n1, P1.p1), d2 = Dot(n2, P2.p1), d3 = Dot(n3, P3.p1);
426     const Point3 n2xn3 = Cross(n2, n3), n3xn1 = Cross(n3, n1), n1xn2 = Cross(n1, n2);
427     const T denom = Dot(n1, n2xn3);
428     if (fabs1(denom) <= EPS) return false; // 平行/共面 → 无唯一交点
429     X = (d1 * n2xn3 + d2 * n3xn1 + d3 * n1xn2) / denom;
430     return true;
431 }
432
433 // ===== 三角形: 面积 / 点包含 / 最近点 / 相交 =====
434 struct Triangle { Point3 a, b, c; };
435
436 inline T triangle_area(const Triangle & t) {
437     return 0.5L * Cross(t.b - t.a, t.c - t.a).len();

```



```

438 }
439
440 inline bool point_in_triangle(const Triangle & tr, const Point3 & p) {
441     // 点在三角形（含边）内：同向法向 + 半平面测试
442     const Point3 n = Cross(tr.b - tr.a, tr.c - tr.a);
443     if (n.len() <= EPS) return false; // 退化三角形
444     const Point3 n1 = Cross(tr.b - tr.a, p - tr.a);
445     const Point3 n2 = Cross(tr.c - tr.b, p - tr.b);
446     const Point3 n3 = Cross(tr.a - tr.c, p - tr.c);
447     if (Dot(n, n1) < -EPS || Dot(n, n2) < -EPS || Dot(n, n3) < -EPS) return false;
448     return true;
449 }
450
451 inline pair<Point3, T> closest_point_on_triangle(const Triangle & tr, const Point3 &
p) {
452     // 点到三角形最近点（含边界）。引用 Ericson 《Real-Time Collision Detection》套路。
453     const Point3 A = tr.a, B = tr.b, C = tr.c;
454     const Point3 AB = B - A, AC = C - A, AP = p - A;
455     const T d1 = Dot(AB, AP), d2 = Dot(AC, AP);
456     if (d1 <= 0 && d2 <= 0) return { A, (p - A).len() };
457
458     const Point3 BP = p - B; const T d3 = Dot(AB, BP), d4 = Dot(AC, BP);
459     if (d3 >= 0 && d4 <= d3) return { B, (p - B).len() };
460
461     const T vc = d1 * d4 - d3 * d2;
462     if (vc <= 0 && d1 >= 0 && d3 <= 0) {
463         const T v = d1 / (d1 - d3); const Point3 Q = A + AB * v; return { Q, (p -
Q).len() };
464     }
465
466     const Point3 CP = p - C; const T d5 = Dot(AB, CP), d6 = Dot(AC, CP);
467     if (d6 >= 0 && d5 <= d6) return { C, (p - C).len() };
468
469     const T vb = d5 * d2 - d1 * d6;
470     if (vb <= 0 && d2 >= 0 && d6 <= 0) {
471         const T w = d2 / (d2 - d6); const Point3 Q = A + AC * w; return { Q, (p -
Q).len() };
472     }
473
474     const T va = d3 * d6 - d5 * d4;
475     if (va <= 0 && (d4 - d3) >= 0 && (d5 - d6) >= 0) {
476         const Point3 BC = C - B; const T w = (d4 - d3) / ((d4 - d3) + (d5 - d6));
const Point3 Q = B + BC * w; return { Q, (p - Q).len() };
477     }
478
479     const Plane P { A, B, C }; const Point3 Q = P.project(p); return { Q, (p -
Q).len() };
480 }
481
482 inline int ray_triangle_intersection(const Ray3 & R, const Triangle & tr, T & t,
Point3 & P) {
483     // 射线与三角形相交（Möller-Trumbore）。返回 0 不交 / 1 命中 (t >= 0)。
484     static const T EPS_MT = 1e-12L;

```

```

485     const Point3 e1 = tr.b - tr.a, e2 = tr.c - tr.a;
486     const Point3 p = Cross(R.d, e2);
487     const T det = Dot(e1, p);
488     if (fabs1(det) <= EPS_MT) return 0; // 平行或共面
489     const T inv = 1.0L / det;
490     const Point3 s = R.o - tr.a;
491     const T u = Dot(s, p) * inv; if (u < -EPS_MT || u > 1 + EPS_MT) return 0;
492     const Point3 q = Cross(s, e1);
493     const T v = Dot(R.d, q) * inv; if (v < -EPS_MT || u + v > 1 + EPS_MT) return 0;
494     t = Dot(e2, q) * inv; if (t < -EPS_MT) return 0;
495     P = R.o + R.d * t; return 1;
496 }
497
498 inline int segment_triangle_intersection(const Segment3 & S, const Triangle & tr,
Point3 & P) {
499     // 线段与三角形相交 (命中点 P)。返回 0/1。
500     const Ray3 R { S.p1, S.p2 - S.p1 };
501     T t; if (!ray_triangle_intersection(R, tr, t, P)) return 0;
502     const T seg2 = (S.p2 - S.p1).len2(); if (seg2 <= EPS) return 0;
503     const T tt = Dot(P - S.p1, S.p2 - S.p1) / seg2;
504     return (tt >= -EPS && tt <= 1 + EPS) ? 1 : 0;
505 }
506
507 inline int triangle_plane_intersection(const Triangle & tr, const Plane & P, Segment3
& out) {
508     // 三角形与平面相交为线段: 返回 1; 共面返回 -1; 不相交返回 0。
509     const T d1 = P.signed_distance(tr.a);
510     const T d2 = P.signed_distance(tr.b);
511     const T d3 = P.signed_distance(tr.c);
512     const int s1 = sgn(d1), s2 = sgn(d2), s3 = sgn(d3);
513     if (s1 == 0 && s2 == 0 && s3 == 0) return -1;
514     if ((s1 >= 0 && s2 >= 0 && s3 >= 0) || (s1 <= 0 && s2 <= 0 && s3 <= 0)) return 0;
515
516     auto cut = [&](const Point3 & X, const Point3 & Y, T dx, T dy, Point3 & I, bool &
ok) {
517         if ((dx > 0 && dy > 0) || (dx < 0 && dy < 0)) { ok = false; return; }
518         const T t = dx / (dx - dy); I = X + (Y - X) * t; ok = true;
519     };
520
521     Point3 I1, I2; bool ok1 = false, ok2 = false;
522     cut(tr.a, tr.b, d1, d2, I1, ok1);
523     cut(tr.b, tr.c, d2, d3, I2, ok2);
524     if (!ok1) cut(tr.c, tr.a, d3, d1, I1, ok1);
525     if (!ok2) cut(tr.c, tr.a, d3, d1, I2, ok2);
526     if (ok1 && ok2) { out = Segment3{ I1, I2 }; return 1; }
527     return 0;
528 }
529
530 // ===== 多边形面积 / 网格体积 =====
531 inline T polygon_area_on_plane(const vector<Point3> & poly) {
532     // 共面多边形面积 (无需显式投影):  $0.5 * |\sum (P_i \times P_{i+1})|$ 
533     const int m = (int)poly.size(); if (m < 3) return 0;
534     Point3 acc { 0, 0, 0 };

```

```

535     for (int i = 0; i < m; ++i) acc = acc + Cross(poly[i], poly[(i + 1) % m]);
536     return 0.5L * acc.len();
537 }
538
539 inline T signed_volume6(Point3 A, Point3 B, Point3 C, Point3 D) {
540     // 四面体有向体积 x6: Cross(B - A, C - A) · (D - A)
541     return Cross(B - A, C - A) * (D - A);
542 }
543
544 inline T mesh_volume_from_triangles(const vector<Triangle> & faces) {
545     // 三角网格体积 (原点分解法; 若面朝向不一致, 取绝对值求和可自救)
546     T v6 = 0;
547     for (const auto & f : faces) v6 += signed_volume6({ 0, 0, 0 }, f.a, f.b, f.c);
548     return fabs1(v6) / 6.0L;
549 }
550
551 // ===== 射线 - 平面 / 球 =====
552 inline int ray_plane_intersection(const Ray3 & R, const Plane & P, T & t, Point3 & I)
553 {
554     // 射线与平面: 返回 -1 共面; 0 平行不交; 1 命中 (t >= 0)
555     const Point3 n = pvec(P);
556     const T denom = Dot(n, R.d);
557     if (sgn(denom) == 0) {
558         if (sgn(Dot(n, R.o - P.p1)) == 0) return -1;
559         return 0;
560     }
561     t = Dot(n, P.p1 - R.o) / denom;
562     if (t < -EPS) return 0;
563     I = R.o + R.d * t; return 1;
564 }
565
566 inline int ray_sphere_intersection(const Ray3 & R, const Sphere & S, array<Point3, 2>
567 & pts) {
568     // 射线与球: 返回 0/1/2 (仅 t >= 0 的解)
569     const Point3 m = R.o - S.c;
570     const T A = Dot(R.d, R.d);
571     const T B = 2.0L * Dot(m, R.d);
572     const T C = Dot(m, m) - S.r * S.r;
573     if (fabs1(A) <= EPS) { if (C > EPS) return 0; pts[0] = R.o; return 1; }
574     const T D = B * B - 4 * A * C; if (D < -EPS) return 0;
575     if (fabs1(D) <= EPS) { const T t = (-B) / (2 * A); if (t < -EPS) return 0; pts[0]
576 = R.o + R.d * t; return 1; }
577     const T sD = safe_sqrt(D);
578     const T t1 = (-B - sD) / (2 * A), t2 = (-B + sD) / (2 * A);
579     int cnt = 0; if (t1 >= -EPS) pts[cnt++] = R.o + R.d * t1; if (t2 >= -EPS)
580 pts[cnt++] = R.o + R.d * t2; return cnt;
581 }
582
583 // ===== 球 / 圆 (3D) 及相关公式 =====
584 struct Circle3 { Point3 c, n; T r; }; // n 需为单位法向
585
586 inline T sphere_surface_area(T R) { return 4.0L * PI * R * R; }
587 inline T sphere_volume(T R) { return (4.0L / 3.0L) * PI * R * R * R; }

```

```

584 inline T spherical_shell_volume(T Rout, T Rin) { return sphere_volume(Rout) -
sphere_volume(Rin); }
585 inline T spherical_shell_surface_area_outer(T Rout) { return
sphere_surface_area(Rout); }
586 inline T spherical_shell_surface_area_inner(T Rin) { return
sphere_surface_area(Rin); }
587
588 inline int line_sphere_intersection(const Line3 & L, const Sphere & S, array<Point3,
2> & pts) {
589     // 直线与球: 返回 0/1/2 (不限制参数范围)
590     const Point3 p = L.p1, d = L.p2 - L.p1, m = p - S.c;
591     const T A = Dot(d, d), B = 2.0L * Dot(m, d), C = Dot(m, m) - S.r * S.r;
592     if (fabs1(A) <= EPS) { if (C > EPS) return 0; pts[0] = p; return 1; }
593     const T D = B * B - 4 * A * C; if (D < -EPS) return 0;
594     if (fabs1(D) <= EPS) { const T t = (-B) / (2 * A); pts[0] = p + d * t; return 1;
}
595     const T sqrtD = safe_sqrt(D);
596     const T t1 = (-B - sqrtD) / (2 * A), t2 = (-B + sqrtD) / (2 * A);
597     pts[0] = p + d * t1; pts[1] = p + d * t2; return 2;
598 }
599
600 inline int plane_sphere_intersection(const Plane & P, const Sphere & S, Circle3 &
out) {
601     // 平面与球相交: 返回 1 圆; -1 相切 (圆退化为点); 0 不相交
602     if (!P.valid()) return 0;
603     const T delta = P.signed_distance(S.c), ad = fabs1(delta);
604     if (ad > S.r + EPS) return 0;
605     const Point3 n = P.unit_normal();
606     const Point3 cc = S.c - n * delta; // 球心到平面的投影
607     const T rr = safe_sqrt(S.r * S.r - ad * ad);
608     out = Circle3{ cc, n, rr };
609     return (fabs1(rr) <= EPS ? -1 : 1);
610 }
611
612 inline T cap_area(T R, T h) { h = clampT(h, (T)0, (T)2 * R); return 2.0L * PI * R *
h; }
613 inline T cap_volume(T R, T h) { h = clampT(h, (T)0, (T)2 * R); return PI * h * h * (R
- h / 3.0L); }
614
615 inline pair<T, T> spherical_cap_on_normal_side(const Sphere & S, const Plane & P) {
616     // 取“法向一侧”的球冠: delta = signed_distance(S.c), h = clamp(R + delta, 0, 2R)
617     const T delta = P.signed_distance(S.c);
618     const T h = clampT(S.r + delta, (T)0, (T)2 * S.r);
619     return { cap_area(S.r, h), cap_volume(S.r, h) };
620 }
621
622 inline pair<T, T> spherical_zone_between_parallel_planes(const Sphere & S, const
Plane & P, const Plane & Q) {
623     // 球带 (两平行平面之间): 面积 = 2πR(h2 - h1), 体积 = vcap(h2) - vcap(h1)
624     if (!parallel(P, Q)) return { 0, 0 };
625     Point3 nP = P.unit_normal(), nQ = Q.unit_normal();
626     Plane QQ = Q; if (Dot(nP, nQ) < 0) std::swap(QQ.p2, QQ.p3); // 统一法向方向
627     T h1 = clampT(S.r + P.signed_distance(S.c), (T)0, (T)2 * S.r);

```

```

628     T h2 = clampT(S.r + QQ.signed_distance(S.c), (T)0, (T)2 * S.r);
629     if (h2 < h1) std::swap(h1, h2);
630     const T A = 2.0L * PI * S.r * (h2 - h1);
631     const T V = cap_volume(S.r, h2) - cap_volume(S.r, h1);
632     return { A, V };
633 }
634
635 inline int sphere_sphere_intersection_circle(const Sphere & A, const Sphere & B,
Circle3 & out) {
636     // 两球相交的交圆: 返回 1 正常; 0 不交; -1 包含; -2 重合
637     const Point3 u = B.c - A.c; const T d = u.len();
638     if (d <= EPS) { if (fabs1(A.r - B.r) <= EPS) return -2; return -1; }
639     if (d > A.r + B.r + EPS) return 0;
640     if (d < fabs1(A.r - B.r) - EPS) return -1;
641     const Point3 n = u / d;
642     const T x = (A.r * A.r - B.r * B.r + d * d) / (2 * d);
643     T rho2 = A.r * A.r - x * x; if (rho2 < 0 && rho2 > -EPS) rho2 = 0; if (rho2 < 0)
return 0;
644     const Point3 center = A.c + n * x;
645     out = Circle3{ center, n, safe_sqrt(rho2) };
646     return (out.r <= EPS ? -1 : 1);
647 }
648
649 inline T sphere_sphere_intersection_volume(T r1, T r2, T d) {
650     // 两球交体积闭式
651     if (d >= r1 + r2 - EPS) return 0;
652     if (d <= fabs1(r1 - r2) + EPS) return sphere_volume(std::min(r1, r2));
653     const T term1 = (r1 + r2 - d);
654     const T num = PI * term1 * term1 * (d * d + 2 * d * (r1 + r2) - 3 * (r1 - r2) *
(r1 - r2));
655     return num / (12.0L * d);
656 }
657 inline T sphere_sphere_intersection_volume(const Sphere & A, const Sphere & B) {
658     return sphere_sphere_intersection_volume(A.r, B.r, (B.c - A.c).len());
659 }
660
661 inline int line_circle3_intersection(const Line3 & L, const Circle3 & C,
array<Point3, 2> & out) {
662     // 直线与空间圆 (圆由圆心/半径/法向描述) 相交: 返回 0/1/2
663     Plane P; (void)plane_from_point_normal(C.c, C.n, P);
664     Point3 X; const int r = Line_cross_plane(L, P, X);
665     if (r == 0) return 0; // 平行不交
666     if (r == 1) { // 与平面仅一交点, 最多与圆 1 点
667         const T d2 = (X - C.c).len2();
668         if (dcmp(d2, C.r * C.r) == 0) { out[0] = X; return 1; }
669         return 0;
670     }
671     // r == -1: 直线在圆平面内 → 平面内“线-圆”2 交/1 交/0 交
672     const T d = Dis_point_line(C.c, L);
673     if (d > C.r + EPS) return 0;
674     Point3 u = dir(L); const T Llen = u.len(); if (Llen <= EPS) return 0; u = u /
Llen;
675     const Point3 H = Point_line_proj(C.c, L);

```

```

676     if (dcmp(d, C.r) == 0) { out[0] = H; return 1; }
677     const T off = safe_sqrt(C.r * C.r - d * d);
678     out[0] = H - u * off; out[1] = H + u * off; return 2;
679 }
680
681 // ===== 共面点的基底 / 2D 映射 / 极角排序 =====
682 struct OrthoBasis { Point3 o, u, v, n; };
683
684 inline OrthoBasis make_plane_basis(const Plane & P) {
685     OrthoBasis B; B.o = P.p1; B.n = P.unit_normal();
686     Point3 a = fabs1(B.n.x) > fabs1(B.n.y) ? Point3{ -B.n.z, 0, B.n.x } : Point3{ 0,
B.n.z, -B.n.y };
687     if (a.len() <= EPS) a = { 1, 0, 0 };
688     B.u = Cross(B.n, a); B.u = B.u / B.u.len();
689     B.v = Cross(B.n, B.u);
690     return B;
691 }
692
693 inline pair<T, T> to2D(const OrthoBasis & B, const Point3 & p) {
694     const Point3 d = p - B.o; return { Dot(d, B.u), Dot(d, B.v) };
695 }
696
697 inline Point3 from2D(const OrthoBasis & B, T X, T Y) { return B.o + B.u * X + B.v *
Y; }
698
699 inline void sort_coplanar_points_ccw(const Plane & P, vector<Point3> & pts) {
700     // 将共面点按 CCW 顺序排序 (相对质心的极角)
701     if (pts.size() <= 2) return;
702     const OrthoBasis B = make_plane_basis(P);
703     vector<pair<T, T>> q(pts.size()); T cx = 0, cy = 0;
704     for (size_t i = 0; i < pts.size(); ++i) { q[i] = to2D(B, pts[i]); cx +=
q[i].first; cy += q[i].second; }
705     cx /= (T)pts.size(); cy /= (T)pts.size();
706     auto ang = [&](const pair<T, T> & a) { return atan2l(a.second - cy, a.first -
cx); };
707     vector<int> id(pts.size()); std::iota(id.begin(), id.end(), 0);
708     std::sort(id.begin(), id.end(), [&](int i, int j) {
709         const T ai = ang(q[i]), aj = ang(q[j]);
710         if (dcmp(ai, aj) != 0) return ai < aj;
711         const T di = (q[i].first - cx) * (q[i].first - cx) + (q[i].second - cy) *
(q[i].second - cy);
712         const T dj = (q[j].first - cx) * (q[j].first - cx) + (q[j].second - cy) *
(q[j].second - cy);
713         return di < dj;
714     });
715     vector<Point3> tmp; tmp.reserve(pts.size()); for (int i : id)
tmp.push_back(pts[i]); pts.swap(tmp);
716 }
717
718 // ===== 平面凸包 (共面点 → 2D Andrew → 回 3D) =====
719 struct P2 { T x, y; };
720 inline int sgn2(T v) { return v > EPS ? 1 : (v < -EPS ? -1 : 0); }

```

```

721 inline T cross2(const P2 & a, const P2 & b, const P2 & c) { return (b.x - a.x) * (c.y
- a.y) - (b.y - a.y) * (c.x - a.x); }
722
723 inline vector<Point3> plane_convex_hull(const vector<Point3> & pts3, const Plane & P)
{
724     // 若点数很少, 直接返回; 否则: 3D→2D 后用 Andrew 单调链求凸包, 再映回 3D
725     if (pts3.size() <= 2) return pts3;
726     const OrthoBasis B = make_plane_basis(P);
727     vector<std::pair<P2, int>> a; a.reserve(pts3.size());
728     for (int i = 0; i < (int)pts3.size(); ++i) { auto t = to2D(B, pts3[i]);
a.push_back({ { t.first, t.second }, i }); }
729     std::sort(a.begin(), a.end(), [&](auto & A, auto & B) { return dcmp(A.first.x,
B.first.x) != 0 ? A.first.x < B.first.x : A.first.y < B.first.y; });
730     a.erase(std::unique(a.begin(), a.end(), [&](auto & A, auto & B) { return
fabs1(A.first.x - B.first.x) <= EPS && fabs1(A.first.y - B.first.y) <= EPS; }),
a.end());
731     if (a.size() <= 2) { vector<Point3> r; for (auto & e : a)
r.push_back(pts3[e.second]); return r; }
732
733     vector<int> st; st.reserve(a.size() * 2);
734     auto push = [&](int idx) {
735         while (st.size() >= 2) {
736             const P2 A = a[st[st.size() - 2]].first, B2 = a[st.back()].first, C =
a[idx].first;
737             if (sgn2(cross2(A, B2, C)) <= 0) st.pop_back(); else break;
738         }
739         st.push_back(idx);
740     };
741
742     for (int i = 0; i < (int)a.size(); ++i) push(i);
743     const int t = (int)st.size();
744     for (int i = (int)a.size() - 2; i >= 0; --i) push(i);
745     st.pop_back();
746
747     vector<Point3> hull; hull.reserve(st.size());
748     for (int id : st) hull.push_back(pts3[a[id].second]);
749     return hull;
750 }
751
752 // ===== 多边形按半空间裁剪 (Sutherland-Hodgman in 3D)
=====
753 inline Point3 segment_plane_intersection_point(const Point3 & A, const Point3 & B,
const Plane & H) {
754     const T da = H.signed_distance(A), db = H.signed_distance(B);
755     if (fabs1(da - db) <= EPS) return A; // 退化兜底
756     const T t = da / (da - db); return A + (B - A) * t;
757 }
758
759 inline vector<Point3> polygon_clip_by_halfspace(const vector<Point3> & poly, const
Plane & H, bool keep_positive = true) {
760     // 输入: 共面多边形 poly; 按平面 H 的一个半空间进行裁剪
761     vector<Point3> ret; if (poly.empty()) return ret;

```

```

762     auto inside = [&](const Point3 & X) { const T d = H.signed_distance(X); return
keep_positive ? d >= -EPS : d <= EPS; };
763     for (size_t i = 0; i < poly.size(); ++i) {
764         const Point3 & A = poly[i];
765         const Point3 & B = poly[(i + 1) % poly.size()];
766         const bool inA = inside(A), inB = inside(B);
767         if (inA && inB) { ret.push_back(B); }
768         else if (inA && !inB) { ret.push_back(segment_plane_intersection_point(A, B,
H)); }
769         else if (!inA && inB) { ret.push_back(segment_plane_intersection_point(A, B,
H)); ret.push_back(B); }
770         else { /* out -> out */ }
771     }
772     // 去重 & 首尾合并
773     if (!ret.empty()) {
774         vector<Point3> tmp; tmp.reserve(ret.size());
775         for (const auto & p : ret) {
776             if (tmp.empty() || (p - tmp.back()).len() > 1e-12L) tmp.push_back(p);
777         }
778         if (tmp.size() >= 2 && (tmp.front() - tmp.back()).len() <= 1e-12L)
tmp.pop_back();
779         ret.swap(tmp);
780     }
781     return ret;
782 }
783
784 // ===== 网格被平面“截一刀” → 截面多边形环（面积/周长）
=====
785 inline vector<Segment3> mesh_plane_cut_segments(const vector<Triangle> & faces, const
Plane & P) {
786     vector<Segment3> segs;
787     for (const auto & f : faces) { Segment3 s; const int r =
triangle_plane_intersection(f, P, s); if (r == 1) segs.push_back(s); }
788     return segs;
789 }
790
791 inline vector<vector<Point3>> stitch_segments_to_loops(const vector<Segment3> & segs,
T merge_eps = 1e-9L) {
792     // 将一组边“拼接成”闭合轮廓（默认数据来自截面，通常可形成若干个环）
793     struct Key { long long x, y, z; };
794     struct Hash { size_t operator () (const Key & k) const { return std::hash<long
long>()(k.x * 1315423911LL ^ k.y * 975311LL ^ k.z * 19260817LL); } };
795     auto key = [&](const Point3 & p) { return Key{ llroundl(p.x / merge_eps),
llroundl(p.y / merge_eps), llroundl(p.z / merge_eps) }; };
796     auto eqp = [&](const Point3 & A, const Point3 & B) { return (A - B).len() <=
merge_eps * 2; };
797
798     std::unordered_map<Key, int, Hash> id;
799     vector<Point3> pts; vector<vector<int>> adj;
800
801     auto get = [&](const Point3 & p) {
802         const Key K = key(p);
803         auto it = id.find(K);

```



```

804         if (it != id.end()) { if (eqp(pts[it->second], p)) return it->second; }
805         const int idx = (int)pts.size(); id[K] = idx; pts.push_back(p);
adj.push_back({}); return idx;
806     };
807
808     for (const auto & s : segs) {
809         const int u = get(s.p1), v = get(s.p2);
810         if (u == v) continue;
811         adj[u].push_back(v);
812         adj[v].push_back(u);
813     }
814
815     vector<int> vis(pts.size(), 0);
816     vector<vector<Point3>> loops;
817
818     for (int st = 0; st < (int)pts.size(); ++st) {
819         if (vis[st] || adj[st].empty()) continue;
820         vector<Point3> poly; poly.reserve(32);
821         int u = st, prev = -1;
822         while (true) {
823             vis[u] = 1; poly.push_back(pts[u]);
824             int nxt = -1; for (int w : adj[u]) if (w != prev) { nxt = w; break; }
825             if (nxt == -1) break; // 开放折线（理论不应发生）
826             prev = u; u = nxt;
827             if (u == st) { loops.push_back(poly); break; }
828         }
829     }
830     return loops;
831 }
832
833 inline void mesh_plane_section(const vector<Triangle> & faces, const Plane & P,
834                               vector<vector<Point3>> & loops, vector<T> & areas,
vector<T> & perims) {
835     // 网格与平面截面：输出各闭合轮廓 + 面积 + 周长
836     const auto segs = mesh_plane_cut_segments(faces, P);
837     loops = stitch_segments_to_loops(segs);
838     areas.clear(); perims.clear();
839     for (auto & poly : loops) {
840         if (poly.size() < 3) { areas.push_back(0); perims.push_back(0); continue; }
841         sort_coplanar_points_ccw(P, poly);
842         areas.push_back(polygon_area_on_plane(poly));
843         T L = 0; for (int i = 0; i < (int)poly.size(); ++i) L += poly[i].dis(poly[(i
+ 1) % poly.size()]);
844         perims.push_back(L);
845     }
846 }
847
848 // ===== 外接：三角形外接圆心（3D）、四面体外接球 =====
849 inline bool triangle_circumcenter(const Point3 & A, const Point3 & B, const Point3 &
C, Point3 & O, T & R) {
850     // 三角形的外接圆心（空间；三点不共线）
851     const Point3 AB = B - A, AC = C - A, n = Cross(AB, AC);
852     const T n2 = n.len2(); if (n2 <= EPS) return false;

```

```

853     const Point3 num = (AB.len2() * Cross(AC, n)) + (AC.len2() * Cross(n, AB));
854     O = A + num / (2 * n2);
855     R = O.dis(A);
856     return true;
857 }
858
859 inline bool tetra_circumsphere(const Point3 & A, const Point3 & B, const Point3 & C,
const Point3 & D, Sphere & S) {
860     // 四面体外接球（四点不共面）。利用三组垂直平分面求交点。
861     auto plane_from_point_normal_local = [&](const Point3 & p, const Point3 & n) {
Plane out; plane_from_point_normal(p, n, out); return out; };
862     const Plane Pab = plane_from_point_normal_local((A + B) / 2.0L, B - A);
863     const Plane Pac = plane_from_point_normal_local((A + C) / 2.0L, C - A);
864     const Plane Pad = plane_from_point_normal_local((A + D) / 2.0L, D - A);
865     Point3 O; if (!plane_plane_plane_intersection(Pab, Pac, Pad, O)) return false;
866     S.c = O; S.r = O.dis(A); return true;
867 }
868
869 // ===== 反射 & 二面角 =====
870 inline Point3 reflect_point_about_plane(const Point3 & P, const Plane & Pl) {
871     // 点关于平面镜像:  $P' = P - 2 * n * (n \cdot (P - p1))$ , n 为单位法向
872     const Point3 n = Pl.unit_normal();
873     const T d = Dot(n, P - Pl.p1);
874     return P - n * (2 * d);
875 }
876
877 inline T dihedral_angle_along_edge(const Point3 & A, const Point3 & B, const Point3 &
C1, const Point3 & C2) {
878     // 沿共享边 AB 的二面角（两相邻三角片 ABC1 与 ABC2, 返回  $[0, \pi]$ ）
879     Point3 n1 = Cross(B - A, C1 - A), n2 = Cross(B - A, C2 - A);
880     if (n1.len() <= EPS || n2.len() <= EPS) return 0;
881     n1 = n1 / n1.len(); n2 = n2 / n2.len();
882     const Point3 e = (B - A) / (B - A).len();
883     const T s = Dot(e, Cross(n1, n2));
884     const T c = clamp_pm1(Dot(n1, n2));
885     return fabs1(atan21(s, c));
886 }
887
888 // ===== 球面多边形面积（大圆边界） =====
889 inline T spherical_polygon_area(const vector<Point3> & Pts, T R) {
890     const int n = (int)Pts.size(); if (n < 3) return 0;
891     vector<Point3> U(n);
892     for (int i = 0; i < n; ++i) { const T L = Pts[i].len(); U[i] = (L <= EPS ? Pts[i]
: Pts[i] / L); }
893     auto tangent_dir = [&](const Point3 & from, const Point3 & at) {
894         Point3 t = from - at * Dot(from, at); const T L = t.len(); return L <= EPS ?
t : t / L;
895     };
896     T sum = 0;
897     for (int i = 0; i < n; ++i) {
898         const int i0 = (i - 1 + n) % n, i1 = i, i2 = (i + 1) % n;
899         const Point3 t1 = tangent_dir(U[i0], U[i1]);
900         const Point3 t2 = tangent_dir(U[i2], U[i1]);

```

```

901     const T c = clamp_pm1(Dot(t1, t2));
902     sum += acos1(c);
903 }
904 const T E = sum - (n - 2) * PI; // 球面超额
905 return fabs1(E) * R * R;
906 }
907
908 // ===== 经纬 ↔ 直角坐标 & 球面两点弧长 =====
909 inline Point3 latlon_to_xyz(T lat_deg, T lon_deg, T r) {
910     // 纬度 lat ∈ [-90, 90], 经度 lon ∈ [-180, 180]
911     const T phi = lat_deg * PI / 180.0L; // φ: 纬
912     const T lam = lon_deg * PI / 180.0L; // λ: 经
913     return { r * cos1(phi) * cos1(lam), r * cos1(phi) * sin1(lam), r * sin1(phi) };
914 }
915
916 inline T Two_Point_Dis_Onball(Point3 p1, Point3 p2, T r) {
917     // 球面两点的大圆距离: r * arccos( (p1·p2) / r^2 )
918     const T t = clamp_pm1((p1 * p2) / (r * r));
919     return r * acos1(t);
920 }
921
922 // ===== 整数点的安全共线 (可选) =====
923 struct Point_int { int x, y, z; };
924 inline bool areCollinear(const Point_int & a, const Point_int & b, const Point_int &
c) {
925     // 用 128 位避免 1e9 量级溢出
926     __int128 abx = (__int128)b.x - a.x, aby = (__int128)b.y - a.y, abz =
(__int128)b.z - a.z;
927     __int128 acx = (__int128)c.x - a.x, acy = (__int128)c.y - a.y, acz =
(__int128)c.z - a.z;
928     return (abx * acy == aby * acx) && (abx * acz == abz * acx) && (aby * acz == abz
* acy);
929 }
930
931
932 // ===== 结束 =====
933 // 提示:
934 // 1) 三角函数前使用 clamp_pm1; sqrt 使用 safe_sqrt, 避免 NaN。
935 // 2) 对所有“单位法向/单位方向”, 请先做非零判断 (已在函数内部处理)。
936 // 3) 输出精度: 可在主程序中使用 std::fixed << std::setprecision(11)。
937
938 // 传入: 起点 A、终点 B (任意尺度, 函数内会单位化)、球半径 r、可接收球面距离 d (弧长单位)
939 // 返回: 球面上可接收区域的面积
940 inline T flight_signal_area(Point3 A, Point3 B, T r, T d) {
941     // --- 单位化 ---
942     if (A.len() <= EPS || B.len() <= EPS) return 0;
943     A = A / A.len(); B = B / B.len();
944
945     // 最短大圆弧的中央角 a ∈ [0, π]
946     const T a = acos1(clamp_pm1(Dot(A, B)));
947
948     // 角半径 δ = d / r
949     T delta = d / r;

```

```

950     if (delta <= 0) return 0;
951     if (delta >= PI) return 4.0L * PI * r * r;
952
953     // 计算单位球面积 S(a, δ)
954     auto unit_stadium_area = [&](T aa, T dd)->T {
955         // 情况1:  $\delta \leq \pi/2$ , 有闭式:  $S = 2 a \sin \delta + 2\pi (1 - \cos \delta)$ 
956         if (dd <= PI / 2 + 1e-15L) {
957             return 2 * aa * sinl(dd) + 2 * PI * (1 - cosl(dd));
958         }
959         // 情况2:  $\delta \geq \pi$ , 整球 (前面已 return)
960         // 情况3:  $\pi/2 < \delta < \pi$ , 用补集  $S = 4\pi - S(a, \pi - \delta)$ ; 若  $2(\pi - \delta) \leq a$  直接整球
961         const T dd2 = PI - dd;
962         if (2 * dd2 <= aa + 1e-15L) return 4 * PI;
963
964         // 数值部分: 计算  $S\_core(a, d)$  ( $d \leq \pi/2$  且  $2d > a$ ) 的积分
965         auto core = [&](T Aang, T Dang)->T {
966             const T cd = cosl(Dang);
967             const T ca2 = cosl(Aang / 2);
968             const T ratio = clamp_pm1(cd / std::max(ca2, (T)EPS));
969             const T lb = asinl(ratio); // 下界纬度
970             const int N = 1024; // Simpson 分段
971             const T dphi = ((PI / 2) - lb) / N;
972
973             auto integrand = [&](T phi)->T {
974                 const T sphl = sinl(phi);
975                 const T arg = clamp_pm1(cd / std::max(sphl, (T)EPS));
976                 return (2 * acosl(arg) - Aang) * sphl; // (L(phi) - a) * sin phi
977             };
978
979             // 按原“正确代码”结构: ret 先放常数项, 再做辛普森
980             T ret = 2 * Dang - Aang;
981             for (int i = 1; i < N; ++i) {
982                 const T phi = lb + dphi * i;
983                 const T cur = integrand(phi);
984                 ret += (i & 1) ? 4 * cur : 2 * cur;
985             }
986             ret *= dphi / 3;
987             ret *= 2;
988             return ret; // 就是 S(a, d) (单位球)
989         };
990
991         // 用补集
992         return 4 * PI - core(aa, dd2);
993     };
994
995     const T unit_area = unit_stadium_area(a, delta);
996     return unit_area * r * r; // 还原到半径 r 的球
997 }
998 /*
999 基础查询 (O(1))
1000 向量与角度: Dot/Cross/Angle;
1001 距离: Dis_point_line/Dis_point_seg/line_line_distance;
1002 投影与反射: Point_line_proj/Plane::project/reflect_point_about_plane;

```

```

1003 共线/共面/平行/垂直判断: coplanar/parallel/vertical。
1004
1005 线/面/三角形相交
1006 直线-平面: Line_cross_plane; 平面-平面: plane_plane_intersection;
1007 三平面交点: plane_plane_plane_intersection;
1008 射线-三角形 (Möller-Trumbore): ray_triangle_intersection; 线段-三角形:
segment_triangle_intersection;
1009 三角形-平面: triangle_plane_intersection。
1010
1011 球/圆相关
1012 线-球/射线-球: line_sphere_intersection/ray_sphere_intersection;
1013 平面-球 (交圆): plane_sphere_intersection;
1014 两球交圆/交体积: sphere_sphere_intersection_circle/sphere_sphere_intersection_volume;
1015 线-空间圆: line_circle3_intersection; 球冠/球带面积体积:
cap_area/cap_volume/spherical_zone_between_parallel_planes。
1016
1017 AABB 快速判交 (碰撞/可视化常用)
1018 线段/射线与 AABB: segment_aabb_intersect/ray_aabb_intersect;
1019 球与 AABB: sphere_aabb_intersect。
1020
1021 三角网格处理 (切片/体积/表面积)
1022 网格体积: mesh_volume_from_triangles;
1023 用平面切网格并输出截面环 + 面积/周长: mesh_plane_cut_segments → stitch_segments_to_loops →
mesh_plane_section;
1024 共面多边形面积: polygon_area_on_plane。
1025
1026 平面凸包 (共面点)
1027 任取平面基底做 3D→2D 映射, 2D Andrew 求凸包再映回:
make_plane_basis/to2D/plane_convex_hull。
1028
1029 球面几何
1030 球面多边形 (大圆边界) 面积: spherical_polygon_area;
1031 经纬↔直角坐标: latlon_to_xyz; 两点球面距离: Two_Point_Dis_Onball;
1032 二面角/折角: dihedral_angle_along_edge。
1033
1034 结合我上一条给你的 flight_signal_area 胶水函数, 可直接解“地球表面加粗大圆弧的覆盖面积” (你那道航班
接收区题)。
1035
1036 */
1037
1038 #include <bits/stdc++.h>
1039 using namespace std;
1040
1041 // 三维凸包模板
1042 // 使用增量法构造, 复杂度  $O(n^2)$ 
1043 // 对于随机点集, 期望复杂度  $O(n \log n)$ 
1044
1045 using T = long long; // 整数坐标
1046 // using T = long double; // 浮点数坐标
1047
1048 const T eps = 0; // 整数用 0
1049 // const T eps = 1e-8; // 浮点数用 1e-8
1050

```

```

1051 // 三维点
1052 struct Point3D {
1053     T x, y, z;
1054     int id = -1;
1055
1056     Point3D() : x(0), y(0), z(0) {}
1057     Point3D(T x, T y, T z) : x(x), y(y), z(z) {}
1058
1059     bool operator==(const Point3D &a) const {
1060         return abs(x - a.x) <= eps && abs(y - a.y) <= eps && abs(z - a.z) <= eps;
1061     }
1062
1063     bool operator<(const Point3D &a) const {
1064         if (abs(x - a.x) > eps) return x < a.x;
1065         if (abs(y - a.y) > eps) return y < a.y;
1066         return z < a.z;
1067     }
1068
1069     Point3D operator+(const Point3D &a) const {
1070         return Point3D(x + a.x, y + a.y, z + a.z);
1071     }
1072
1073     Point3D operator-(const Point3D &a) const {
1074         return Point3D(x - a.x, y - a.y, z - a.z);
1075     }
1076
1077     Point3D operator*(const T k) const {
1078         return Point3D(k * x, k * y, k * z);
1079     }
1080
1081     Point3D operator/(const T k) const {
1082         return Point3D(x / k, y / k, z / k);
1083     }
1084
1085     // 点积
1086     T operator*(const Point3D &a) const {
1087         return x * a.x + y * a.y + z * a.z;
1088     }
1089
1090     // 叉积
1091     Point3D operator^(const Point3D &a) const {
1092         return Point3D(y * a.z - z * a.y, z * a.x - x * a.z, x * a.y - y * a.x);
1093     }
1094
1095     T len2() const {
1096         return x * x + y * y + z * z;
1097     }
1098
1099     // 浮点数专用
1100     long double len() const {
1101         return sqrtl((long double)(x * x + y * y + z * z));
1102     }
1103 };

```

```

1104
1105 // 三维凸包的面
1106 struct Face {
1107     int v[3]; // 三个顶点的下标, 逆时针存储 (从外部看)
1108
1109     Face() {}
1110     Face(int a, int b, int c) {
1111         v[0] = a; v[1] = b; v[2] = c;
1112     }
1113
1114     // 计算面的法向量 (外法向量)
1115     Point3D normal(const vector<Point3D> &p) const {
1116         return (p[v[1]] - p[v[0]]) ^ (p[v[2]] - p[v[0]]);
1117     }
1118
1119     // 判断点 q 是否在面的外侧
1120     // 返回值: > 0 在外侧, < 0 在内侧, = 0 在面上
1121     T point_outside(const vector<Point3D> &p, const Point3D &q) const {
1122         Point3D n = normal(p);
1123         return n * (q - p[v[0]]);
1124     }
1125 };
1126
1127 // 三维凸包
1128 struct ConvexHull3D {
1129     vector<Point3D> p; // 所有点
1130     vector<Face> faces; // 凸包的所有面
1131
1132     ConvexHull3D(vector<Point3D> points) : p(points) {
1133         build();
1134     }
1135
1136     void build() {
1137         int n = p.size();
1138         if (n < 4) {
1139             // 点数不足, 无法构成三维凸包
1140             return;
1141         }
1142
1143         // 随机打乱, 提高期望性能
1144         random_shuffle(p.begin(), p.end());
1145
1146         // 找到初始四面体
1147         // 先找三个不共线的点
1148         vector<int> init;
1149         init.push_back(0);
1150
1151         // 找第二个点 (与第一个点不同)
1152         for (int i = 1; i < n; i++) {
1153             if (!(p[i] == p[0])) {
1154                 init.push_back(i);
1155                 break;
1156             }

```

```

1157     }
1158     if (init.size() < 2) return;
1159
1160     // 找第三个点 (不与前两个点共线)
1161     for (int i = 0; i < n; i++) {
1162         Point3D v1 = p[init[1]] - p[init[0]];
1163         Point3D v2 = p[i] - p[init[0]];
1164         if ((v1 ^ v2).len2() > eps) {
1165             init.push_back(i);
1166             break;
1167         }
1168     }
1169     if (init.size() < 3) return;
1170
1171     // 找第四个点 (不与前三个点共面)
1172     for (int i = 0; i < n; i++) {
1173         Face f(init[0], init[1], init[2]);
1174         if (abs(f.point_outside(p, p[i])) > eps) {
1175             init.push_back(i);
1176             break;
1177         }
1178     }
1179     if (init.size() < 4) return;
1180
1181     // 构造初始四面体
1182     // 确保第四个点在前三个点构成的面的外侧
1183     Face test(init[0], init[1], init[2]);
1184     if (test.point_outside(p, p[init[3]]) < 0) {
1185         swap(init[1], init[2]);
1186     }
1187
1188     faces.push_back(Face(init[0], init[1], init[2]));
1189     faces.push_back(Face(init[0], init[3], init[1]));
1190     faces.push_back(Face(init[1], init[3], init[2]));
1191     faces.push_back(Face(init[2], init[3], init[0]));
1192
1193     // 标记已使用的点
1194     vector<bool> used(n, false);
1195     for (int i : init) used[i] = true;
1196
1197     // 增量添加剩余的点
1198     for (int i = 0; i < n; i++) {
1199         if (used[i]) continue;
1200         add_point(i);
1201     }
1202 }
1203
1204 void add_point(int idx) {
1205     // 找到所有能"看到"点 idx 的面
1206     vector<bool> visible(faces.size(), false);
1207     for (int i = 0; i < faces.size(); i++) {
1208         if (faces[i].point_outside(p, p[idx]) > eps) {
1209             visible[i] = true;

```



```

1210     }
1211 }
1212
1213 // 找到可见面的边界边
1214 map<pair<int, int>, int> edge_count;
1215 for (int i = 0; i < faces.size(); i++) {
1216     if (visible[i]) {
1217         for (int j = 0; j < 3; j++) {
1218             int a = faces[i].v[j];
1219             int b = faces[i].v[(j + 1) % 3];
1220             if (a > b) swap(a, b);
1221             edge_count[{a, b}]++;
1222         }
1223     }
1224 }
1225
1226 // 边界边是只出现一次的边
1227 vector<pair<int, int>> boundary;
1228 for (auto &e : edge_count) {
1229     if (e.second == 1) {
1230         boundary.push_back(e.first);
1231     }
1232 }
1233
1234 // 删除可见面
1235 vector<Face> new_faces;
1236 for (int i = 0; i < faces.size(); i++) {
1237     if (!visible[i]) {
1238         new_faces.push_back(faces[i]);
1239     }
1240 }
1241
1242 // 用新点和边界边构造新的面
1243 for (auto &e : boundary) {
1244     int a = e.first, b = e.second;
1245     // 确定正确的顶点顺序
1246     // 需要找到包含边 (a,b) 的可见面，确定方向
1247     for (int i = 0; i < faces.size(); i++) {
1248         if (!visible[i]) continue;
1249         for (int j = 0; j < 3; j++) {
1250             int v1 = faces[i].v[j];
1251             int v2 = faces[i].v[(j + 1) % 3];
1252             if ((v1 == a && v2 == b) || (v1 == b && v2 == a)) {
1253                 if (v1 == a && v2 == b) {
1254                     new_faces.push_back(Face(a, b, idx));
1255                 } else {
1256                     new_faces.push_back(Face(b, a, idx));
1257                 }
1258                 goto next_edge;
1259             }
1260         }
1261     }
1262     next_edge++;

```

```

1263     }
1264
1265     faces = new_faces;
1266 }
1267
1268 // 获取凸包的所有顶点（去重）
1269 vector<int> get_vertices() const {
1270     set<int> vertex_set;
1271     for (const auto &f : faces) {
1272         for (int i = 0; i < 3; i++) {
1273             vertex_set.insert(f.v[i]);
1274         }
1275     }
1276     return vector<int>(vertex_set.begin(), vertex_set.end());
1277 }
1278
1279 // 计算凸包的表面积（浮点数）
1280 long double surface_area() const {
1281     long double area = 0;
1282     for (const auto &f : faces) {
1283         Point3D n = f.normal(p);
1284         area += n.len() / 2.0;
1285     }
1286     return area;
1287 }
1288
1289 // 计算凸包的体积（浮点数）
1290 long double volume() const {
1291     long double vol = 0;
1292     Point3D origin(0, 0, 0);
1293     for (const auto &f : faces) {
1294         // 四面体体积 = |a · (b × c)| / 6
1295         Point3D a = p[f.v[0]];
1296         Point3D b = p[f.v[1]];
1297         Point3D c = p[f.v[2]];
1298         vol += (long double)(a * (b ^ c)) / 6.0;
1299     }
1300     return abs(vol);
1301 }
1302 };
1303
1304 signed main() {
1305     ios::sync_with_stdio(false);
1306     cin.tie(0);
1307
1308     int n;
1309     cin >> n;
1310
1311     vector<Point3D> points(n);
1312     for (int i = 0; i < n; i++) {
1313         cin >> points[i].x >> points[i].y >> points[i].z;
1314         points[i].id = i;
1315     }

```

```
1316
1317     ConvexHull3D hull(points);
1318
1319     cout << "面数: " << hull.faces.size() << "\n";
1320     cout << "顶点数: " << hull.get_vertices().size() << "\n";
1321     cout << "表面积: " << fixed << setprecision(10) << hull.surface_area() << "\n";
1322     cout << "体积: " << fixed << setprecision(10) << hull.volume() << "\n";
1323
1324     return 0;
1325 }
```


